MANUAL DE INSTALACIÓN

ASIGNATURA:
IA-220 EVALUACIÓN DE SISTEMAS

CATEDRÁTICO:
HECTOR ENRIQUE HERNÁNDEZ PEDRANO

DOCUMENTACIÓN TÉCNICA
MANUAL DE INSTALACIÓN

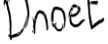
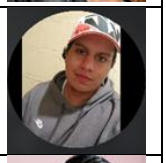
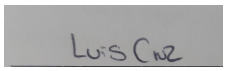
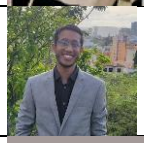
PROYECTO IMPLEMENTADO EN
GRUPO FARINTER

POR EQUIPO DE TRABAJO #3

CIUDAD UNIVERSITARIA

08/07/2025

CONTRIBUYENTES

Elaborado por	Número Cuenta	Firma	Foto
Gabriela Gissel Santos Cruz	20161031752		
Samuel Enrique Godoy Pineda	20161004897		
Diego Noé Espinal Ponce	20201000816		
Luis Armando Cruz Sánchez	20161005645		
Allan Eduardo Suazo Alvarenga	20191005754		
Darwin Alejandro Sánchez Soriano	20191000413		

Revisado y Aprobado por	Área Funcional	Fecha	Firma
Hector Enrique Hernandez Pedrano	Informática Administrativa	**8/7/2025**	

Contenido General

1. INTRODUCCIÓN 4
2. ESPECIFICACIONES TÉCNICAS DEL APLICATIVO 6
2.1 REQUERIMIENTOS TÉCNICOS EN INFRAESTRUCTURA TECNOLÓGICA: 6
2.2 BASE DE DATOS: 7
2.3 TECNOLOGÍA DEL APLICATIVO: 7
2.4 PLATAFORMA PARA LOS REPORTES: 8
3. REQUERIMIENTOS MÍNIMOS/RECOMENDADOS 8
4. LISTADO DE COMPONENTES A INSTALAR 8
4.1 VISUAL CODE 9
4.2 PROYECTO SISMOS 13
4.3 INSTALACIÓN DE XAMPP 17
4.4 INSTALACIÓN DE NODE.JS 21
5. INICIALIZACIÓN DE LA BASE DE DATOS 24
6. REALIZACIÓN UN BACKUP/RESTORE 26
6.1 BACKUP DESDE EL SISTEMA 26
6.2 BACKUP DESDE LA BASE DE DATOS 29
6.3 RESTORE DESDE EL SISTEMA 31
6.4 RESTORE DESDE LA BASE DE DATOS 32
7. SEGURIDAD, PARAMETROS, USO DE ROLES 35
7.1 PARÁMETROS 35
7.2 MANTENIMIENTO DE ROLES 35
7.2.1 Rol de Administrador: 35
7.2.2 Rol de Usuario: 35
7.3 PERMISOS DEL SISTEMA 36
7.4 MATRIZ DE ACCESO AL SISTEMA 36
8. ANEXOS 37
8.1 INSTALACIÓN DE APLICATIVO EN SERVIDOR PRIVADO VIRTUAL (VPS) 37
8.2 PROCESO DE LEVANTAMIENTO VPS 39
8.3 GLOSARIO 45

INDICE DE IMAGENES

FIGURA 1 - VISUAL CODE EN EL NAVEGADOR 9
FIGURA 2 - INICIO DE INSTALACIÓN 10
FIGURA 3 - COMPONENTE DE INSTALACIÓN 11
FIGURA 4 - PROCESO DE INSTALACIÓN 12
FIGURA 5 - BUSCAMOS GITHUB EN EL NAVEGADOR 13
FIGURA 6 - BÚSQUEDA DEL REPOSITORIO SIMOS 13
FIGURA 7 - COPIAR URL 14
FIGURA 8 - INICIALIZACIÓN DE VISUAL CODE 14
FIGURA 9 - CLONAR REPOSITORIO 15
FIGURA 10 - PEGAR URL 15
FIGURA 11 - CARPETA DE DESTINO 16
FIGURA 12 - INICIO DE INSTALACIÓN XAMPP 17
FIGURA 13 - COMPONENTES DE INSTALACIÓN XAMPP 18
FIGURA 14 - CARPETA DE DESTINO DE INSTALACIÓN DE XAMPP 18
FIGURA 15 - DE INSTALACIÓN DE XAMPP 19
FIGURA 16 - FIN DEL PROCESO DE INSTALACIÓN DE XAMPP 19
FIGURA 17 - VISTA GENERAL DE XAMPP 20
FIGURA 18 - BOTONES PARA EJECUTAR LOS DIFERENTES MÓDULOS 20
FIGURA 19 - INSTALACIÓN DE NODE.JS 21
FIGURA 20 - COMPONENTE DE INSTALACIÓN DE NODE.JS 22
FIGURA 21 - INSTALACIÓN DE NODE.JS 22
FIGURA 22 - PROCESO DE INSTALACIÓN DE NODE.JS 23
FIGURA 23 - FIN DEL INSTALACIÓN DE NODE.JS 23
FIGURA 24 - UBICACIÓN DE LA OPCIÓN DE COPIA DE SEGURIDAD. 26
FIGURA 25 - CREACIÓN DE COPIA DE SEGURIDAD 27
FIGURA 26 - BACKUP REALIZADO. 28
FIGURA 27 - VISTA DEL SERVIDOR. 29
FIGURA 28 - MENÚ DE OPCIONES. 29
FIGURA 29 - CAMPOS A LLENAR. 30
FIGURA 30 - BACKUP DE BASE DE DATOS CREADO FORMA MANUAL. 30

FIGURA 31 - PRIMER PASO DE RESTAURACIÓN. 31
FIGURA 32 - CONFIRMACIÓN DE RESTAURACIÓN. 31
FIGURA 33 - RESTAURACIÓN FINALIZADA. 32
FIGURA 34 - BASE DE DATOS DONDE SE RESTAURARÁ. 32
FIGURA 35 - OPCIÓN DE RESTAURAR 33
FIGURA 36 - SELECCIÓN DE ARCHIVO BACKUP. 33
FIGURA 37 - ÚLTIMO PASO PARA LA RESTAURACIÓN. 34
FIGURA 38 - RESTAURACIÓN TERMINADA. 34
FIGURA 38 - INGRESO AL SERVIDOR PRIVADO VIRTUAL (VPS) 37
FIGURA 39 - ERROR DEVUELTO BASE DE DATOS 38
FIGURA 40 - VERIFICACIÓN DE BASE DE DATOS ACTIVA 38
FIGURA 41 - VERIFICACIÓN DE BASE DE DATOS EXISTENTE 38
FIGURA 42 - ERROR SE MANTIENE 39
FIGURA 43 - IMPORTACIÓN DE PROYECTO 39
FIGURA 44 - NAVEGACIÓN EN LA CARPETA SIMOS 40
FIGURA 45 - LOCALIZACIÓN DE VARIABLE AUTH_SECRET 40
FIGURA 46 - CONFIRMACIÓN 40
FIGURA 47 - CONFIRMACIÓN FINAL 41
FIGURA 48 - EJECUCIÓN DE COMANDO NPM INSTALL 41
FIGURA 49 - LEVANTAMIENTO DE BASE DE DATOS 41
FIGURA 50 - MODIFICACIÓN DE ARCHIVO .ENV 42
FIGURA 51 - EJECUCIÓN DE COMANDO NPM RUN BUILD 43
FIGURA 52 - LEVANTAMIENTO DEL SERVICIO 44
FIGURA 53 - SERVIDOR LEVANTADO 44

1. INTRODUCCIÓN

El presente manual técnico tiene como objetivo proporcionar una guía detallada para la instalación, configuración, operación del (SIMOS). Este sistema ha sido diseñado para supervisar en tiempo real el estado y rendimiento de servicios críticos dentro de Grupo Farinter, permitiendo la detección temprana de incidencias, la mejora en los tiempos de respuesta y la optimización.

El Sistema de Monitoreo de Servicios centraliza la información proveniente de servidores y servicios, ofreciendo visibilidad mediante paneles gráficos, alertas. Gracias a estas funcionalidades, se puede actuar de manera proactiva ante posibles interrupciones o fallas del servicio.

Este manual está dirigido a administradores de sistemas, personal de soporte técnico, ingenieros de operaciones y cualquier usuario responsable del monitoreo y la continuidad operativa de los servicios tecnológicos. A lo largo del documento se describen los requisitos técnicos del sistema, requerimientos mínimos, componentes a instalar, etc.

2. ESPECIFICACIONES TÉCNICAS DEL APLICATIVO

2.1 REQUERIMIENTOS TÉCNICOS EN INFRAESTRUCTURA TECNOLÓGICA:

Característica	Recurso Mínimos	Recurso Recomendado
Sistema Operativo	*Windows 10 / Ubuntu 20.04*	Windows 11 / Ubuntu 22.04
Procesador (CPU)	Intel Core i3 o equivalente	Intel Core i5 o superior / AMD Ryzen 5 o superior
Memoria RAM	4 GB	8 GB o más
Almacenamiento	10 GB de espacio libre	SSD con al menos 20 GB libres
Editor de Código	Visual Studio Code (última versión estable)	Visual Studio Code con extensiones recomendadas (Prettier, Slint, Prisma, etc.)
Navegador Web	Google Chrome / Firefox actualizado	Google Chrome actualizado
Node.js	Versión 18.x	Versión LTS más reciente
Gestor de Paquetes	npm	npm

2.2 BASE DE DATOS:

Descripción	Arquitectura y Versión	Configuración
Motor de base de datos	PostgreSQL v14 o superior	Configuración estándar para entorno de desarrollo y producción
ORM utilizado	Prisma ORM (última versión estable)	Generación automática de esquemas con migraciones versionadas
Modelo de datos	Esquema relacional	Normalizado con claves primarias y foráneas. Integridad referencial garantizada
Conexión con backend	Prisma Client generado automáticamente	Uso de .env para gestionar credenciales y variables de conexión

2.3 TECNOLOGÍA DEL APLICATIVO:

Descripción	Arquitectura y Versión	Configuración
HTML	HTML 5.2	Instalar Visual Studio Code como entorno de trabajo. Crear una estructura básica de archivos.
CSS	Versión 3	
JavaScript	ECMAScript 2022	
Node JS	Node.js v 22.11.0	Obtenga más información sobre lo que Node.js puede ofrecer con nuestros materiales de aprendizaje.
Next.js	Next.js v13+	Configurado con rutas automáticas, páginas, APIs, y archivos de configuración (next.config.js).
TypeScript	TypeScript 5+	Configuración en tsconfig.json; usado en todo el código fuente para prevenir errores en tiempo real.

2.4 PLATAFORMA PARA LOS REPORTES:

Descripción	Arquitectura y Versión	Configuración
Exportación de reportes en PDF	jsPDF 2.5.1	Plantillas con logotipo, fecha y datos tabulados. Integrado en el frontend.

3. REQUERIMIENTOS MÍNIMOS/RECOMENDADOS

No.	Requerimiento	Mínimo	Recomendado
1	Procesador	1 núcleos	2 Núcleos
2	Memoria RAM	6 GB	16
3	Espacio en disco	20 GB	100 GB
4	Sistema Operativo	LINUX O WINDOWS	Ubuntu 24.04 LTS
5	Entre Otros		

4. LISTADO DE COMPONENTES A INSTALAR

No.	Nombre Componente	Versión
1	Visual code	1.102.2
2	VPS Ubuntu	24.04
3	Proyecto SIMOS	1.0
4	PostgreSQL	17
5	NODE.JS	22.16.0

4.1 VISUAL CODE

- Al ingresar a la página web oficial de Visual Studio Code (https://code.visualstudio.com/), los usuarios pueden visualizar la pantalla principal de la aplicación.

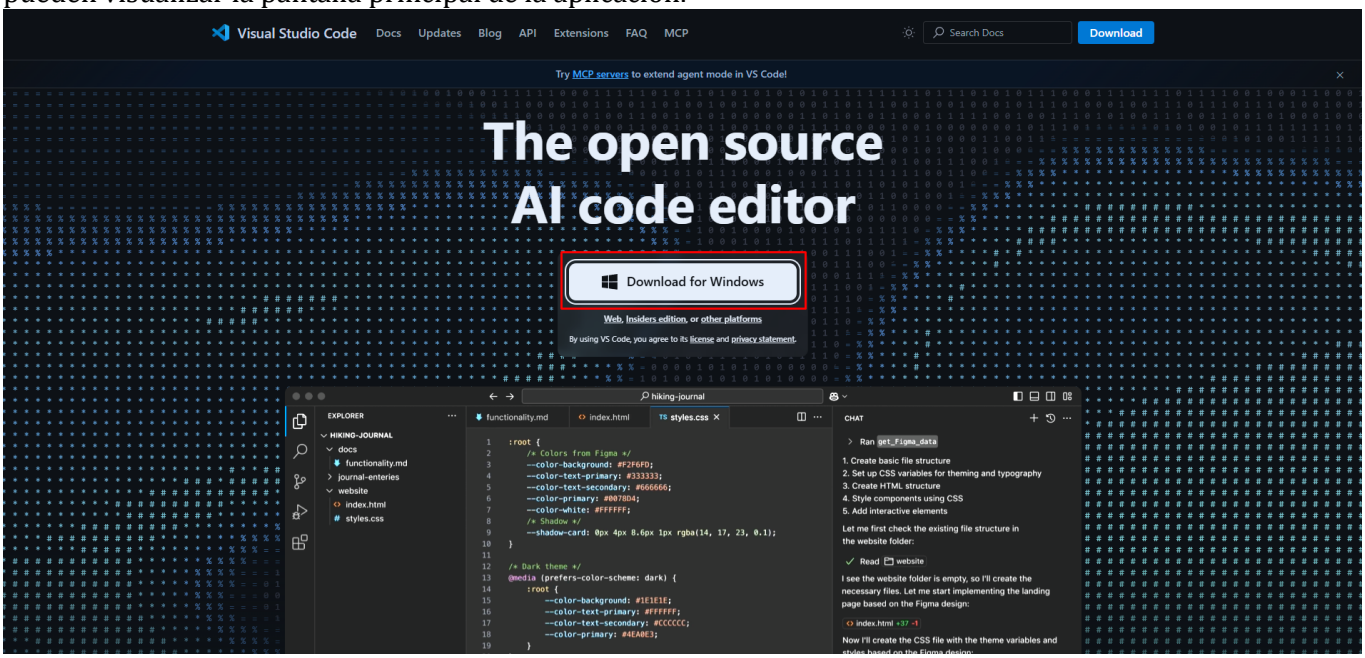


FIGURA 1 - VISUAL CODE EN EL NAVEGADOR

- Los usuarios deben hacer clic en el recuadro que indica "Download for Windows". Esta acción iniciará la descarga de un archivo, el cual deberá ser ejecutado para proceder con la instalación. No se requiere ninguna configuración especial durante este proceso.

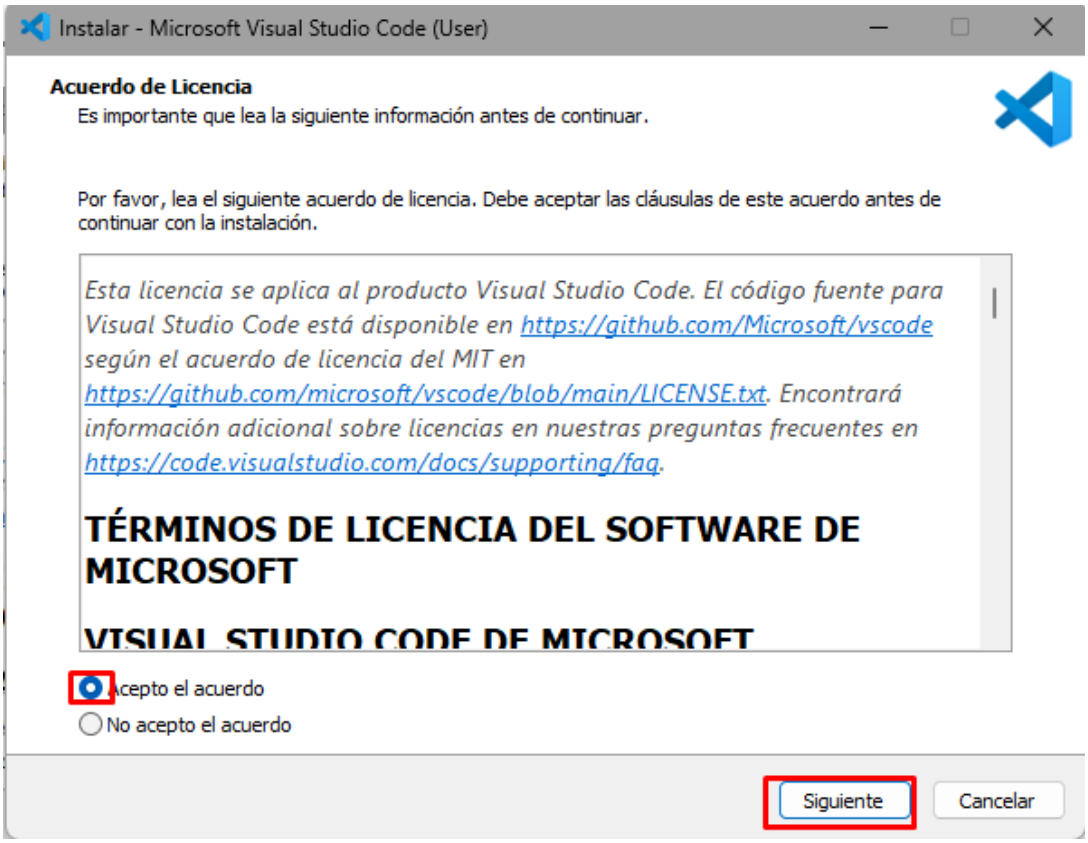


FIGURA 2 - INICIO DE INSTALACIÓN

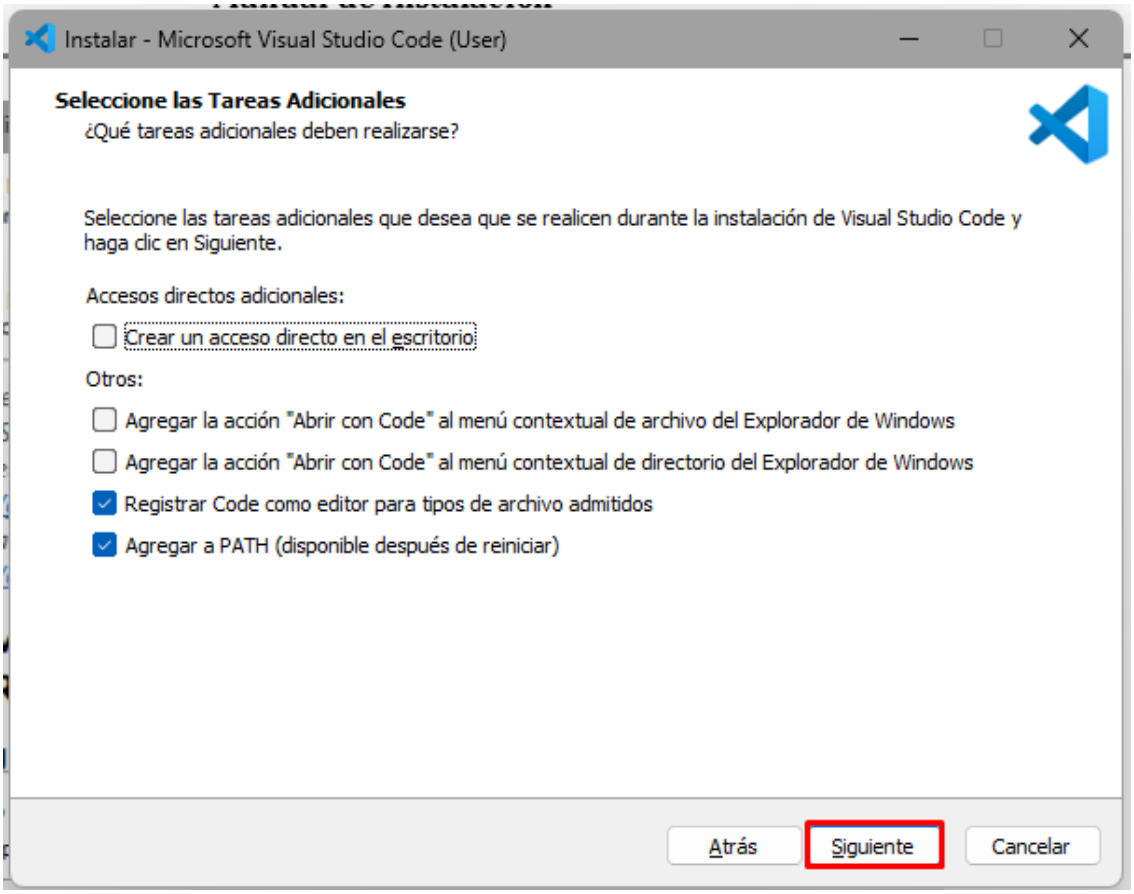


FIGURA 3 - COMPONENTE DE INSTALACIÓN

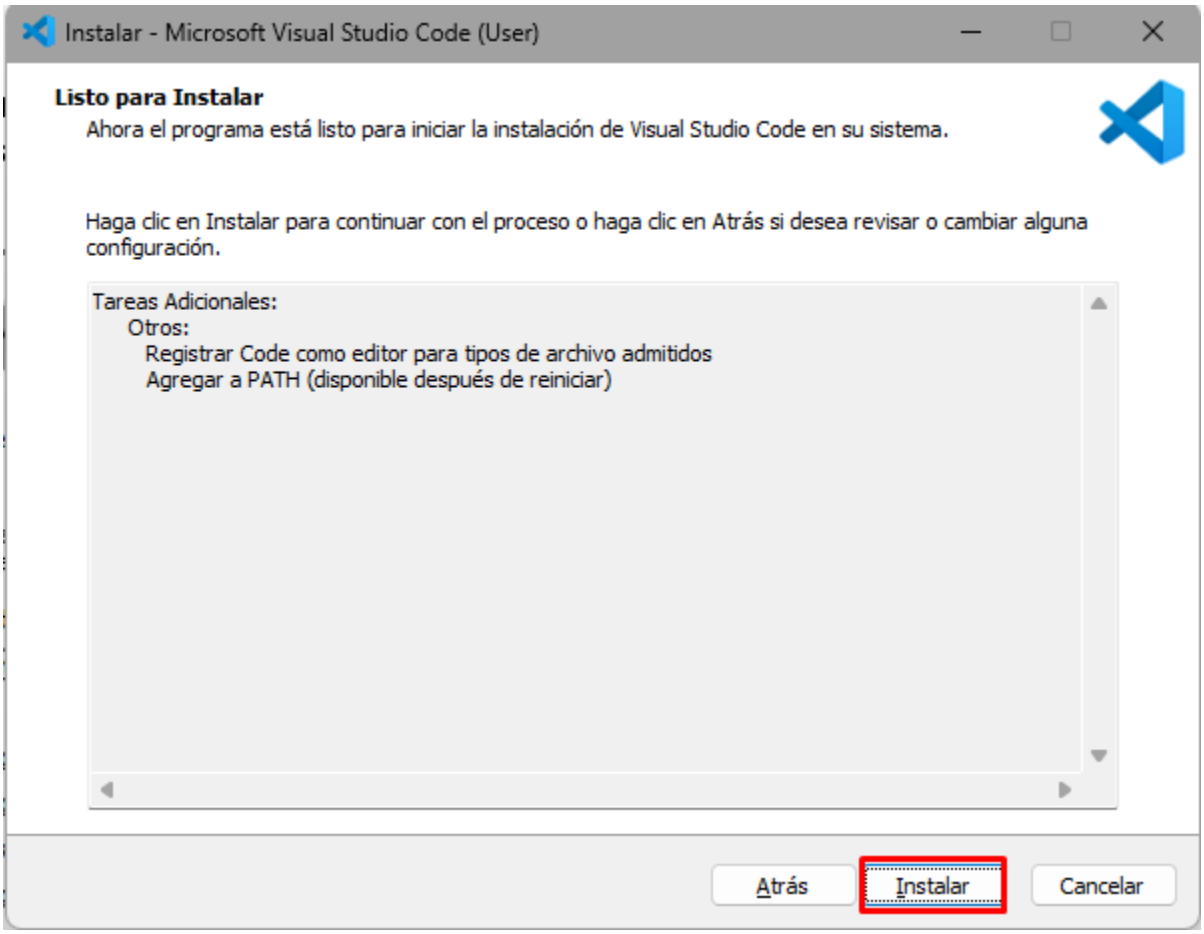


FIGURA 4 - PROCESO DE INSTALACIÓN

4.2 PROYECTO SISMOS

- Para iniciar la clonación del proyecto "Simos" en Visual Studio Code, los usuarios deben abrir GitHub en el navegador web. Este constituye el primer paso para acceder al repositorio del proyecto.

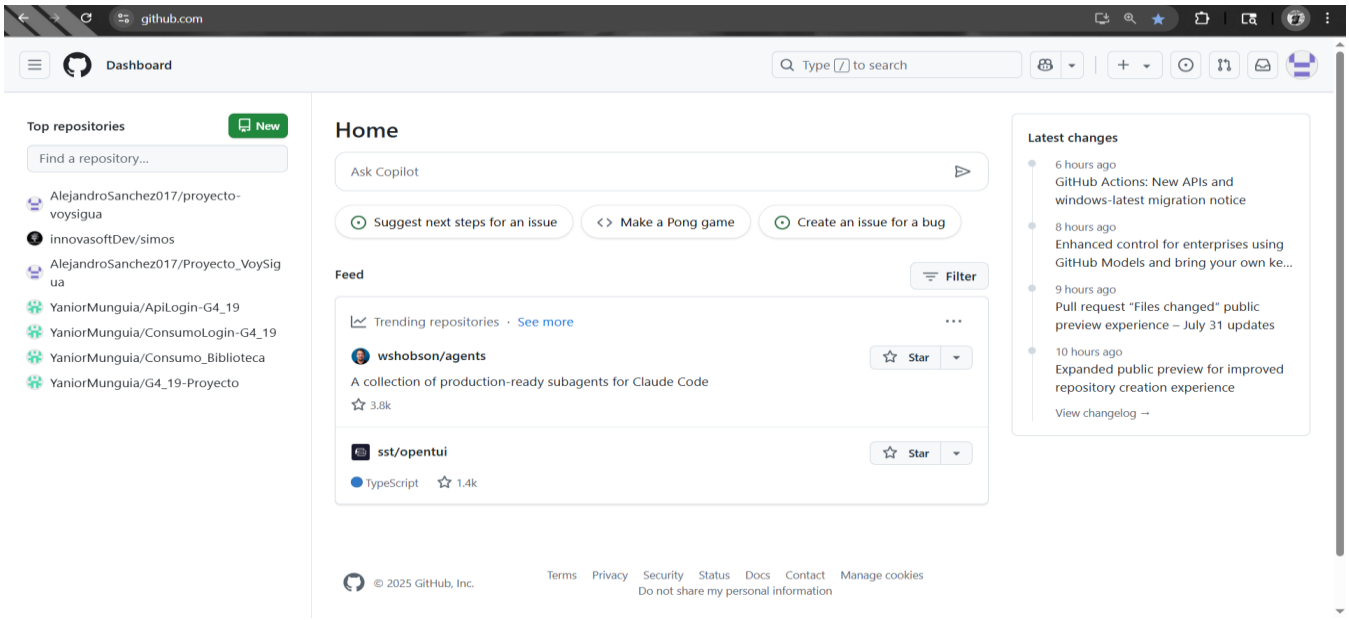


FIGURA 5 - BUSCAMOS GITHUB EN EL NAVEGADOR

- A continuación, **se debe buscar el repositorio del proyecto "Simos"** dentro de la interfaz de GitHub.

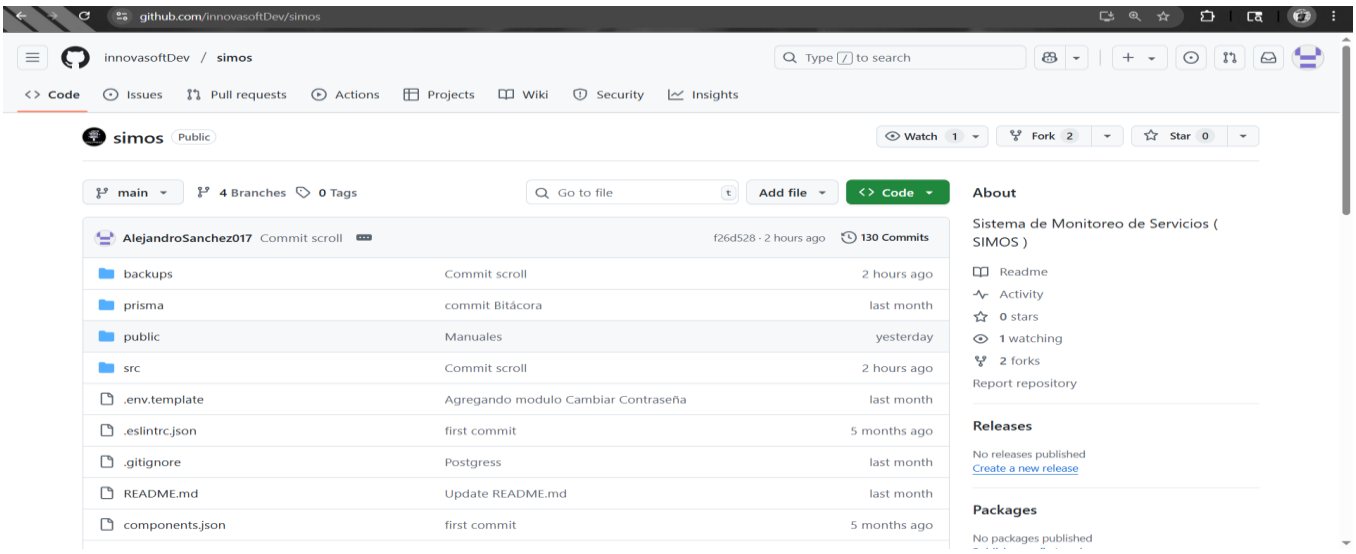


FIGURA 6 - BÚSQUEDA DEL REPOSITORIO SIMOS

- Luego, **se debe hacer clic en la opción etiquetada como "Code"**. Esto abrirá una ventana emergente desde la cual **se deberá copiar la URL proporcionada**.

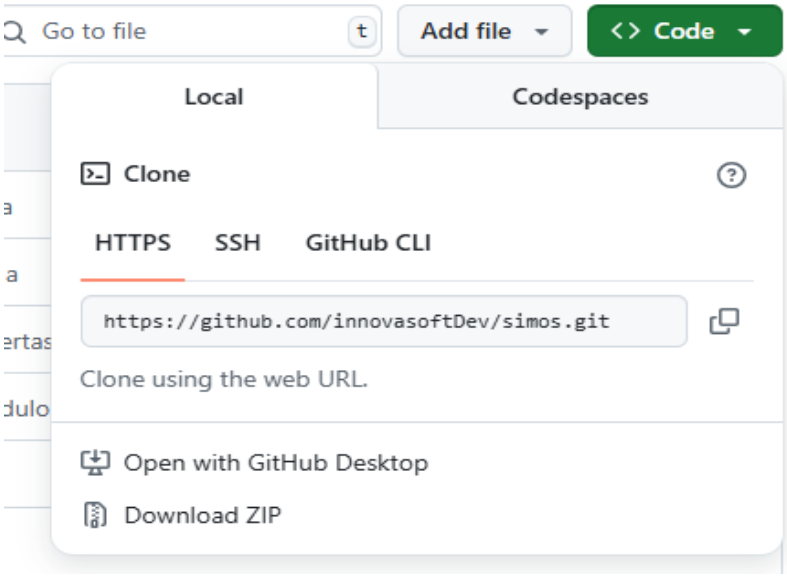


FIGURA 7 - COPIAR URL

- Posteriormente, **se debe abrir Visual Studio Code** y, una vez dentro, **hacer clic en la opción "Explorer"**. Esto desplegará una ventana donde **se deberá seleccionar "Clone Repository"**.

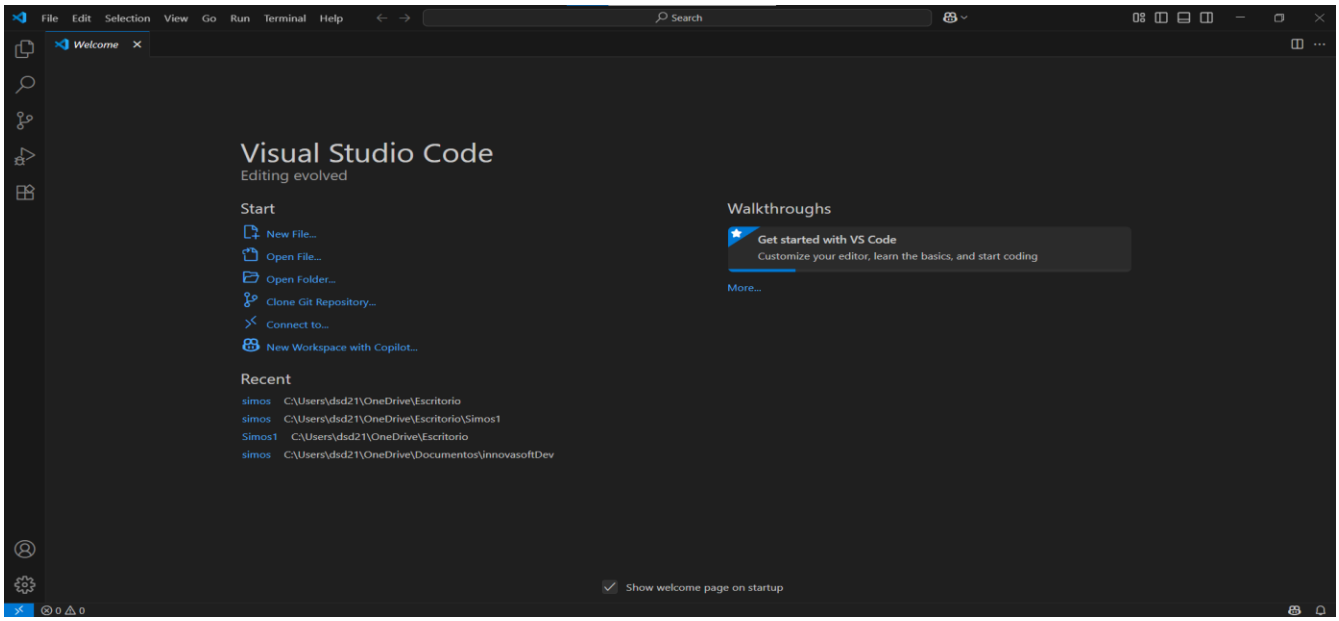


FIGURA 8 - INICIALIZACIÓN DE VISUAL CODE

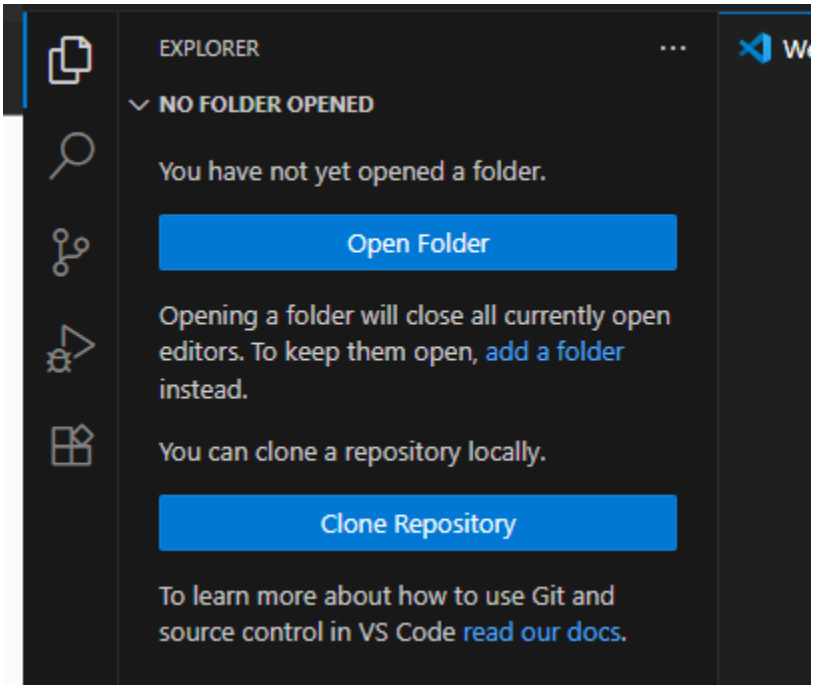


FIGURA 9 - CLONAR REPOSITORIO

- A continuación, **se abrirá una ventana donde se deberá pegar la URL** previamente copiada y **presionar Enter** para iniciar el proceso de clonación.

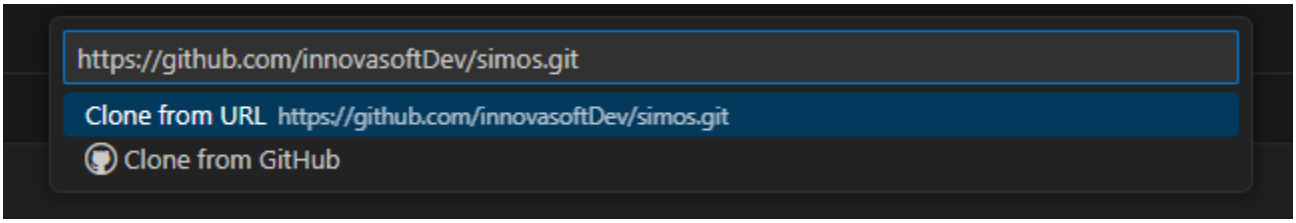


FIGURA 10 - PEGAR URL

- A continuación, **se abrirá el explorador de archivos** para que el usuario **indique en qué carpeta desea clonar el repositorio**.

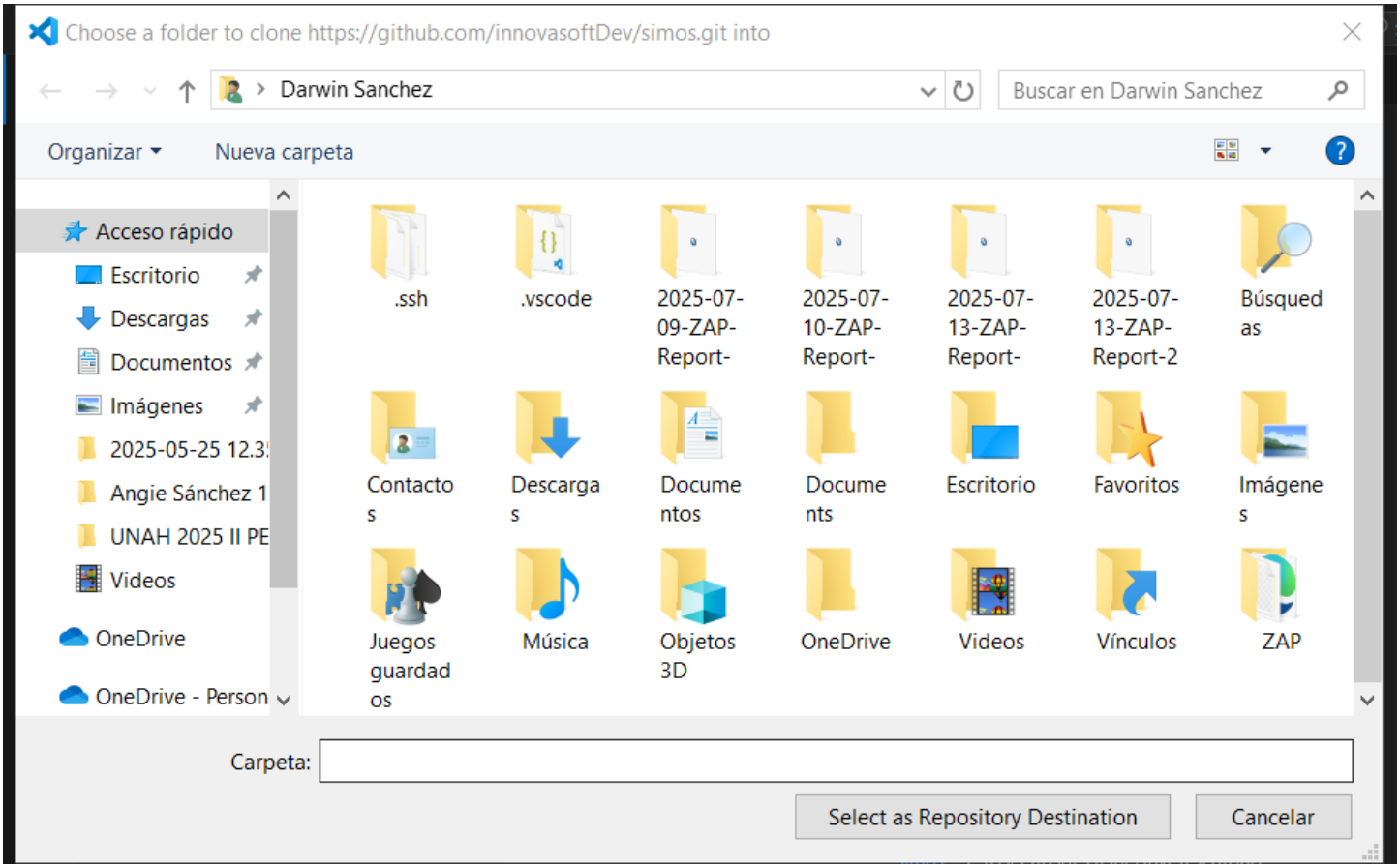


FIGURA 11 - CARPETA DE DESTINO

- Después de seleccionar la carpeta deseada, **se debe hacer clic en "Select as Repository Destination"**. Con esto, el repositorio será clonado exitosamente.

4.3 INSTALACIÓN DE XAMPP

- **Se descarga y ejecuta el archivo .exe desde la página de descarga de XAMPP**. Al abrirse la nueva ventana de instalación, **se debe hacer clic en "Next"**.

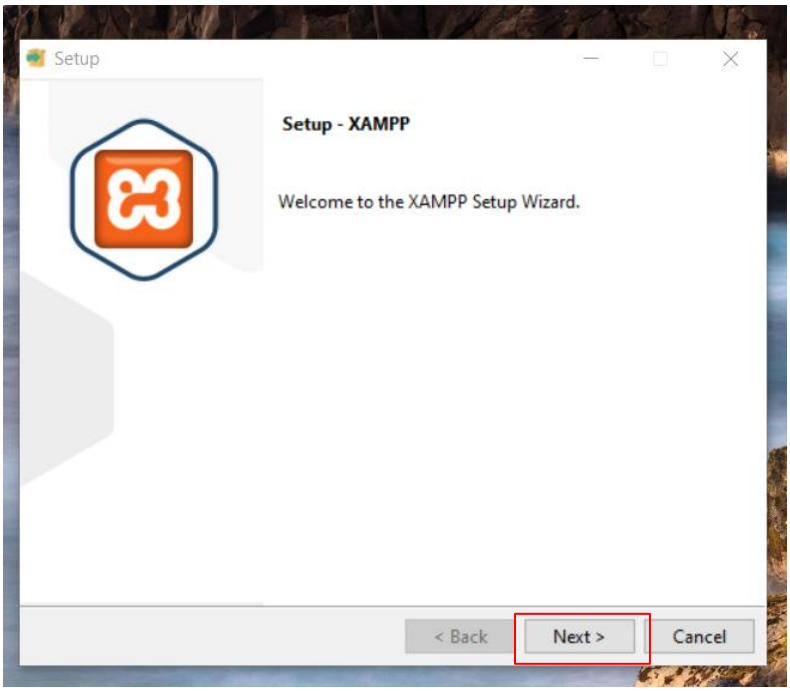


FIGURA 12 - INICIO DE INSTALACIÓN XAMPP

- **Se abrirá una ventana subsiguiente que mostrará los componentes disponibles para instalar**. Una vez revisados, **se debe hacer clic en "Next"** para continuar.

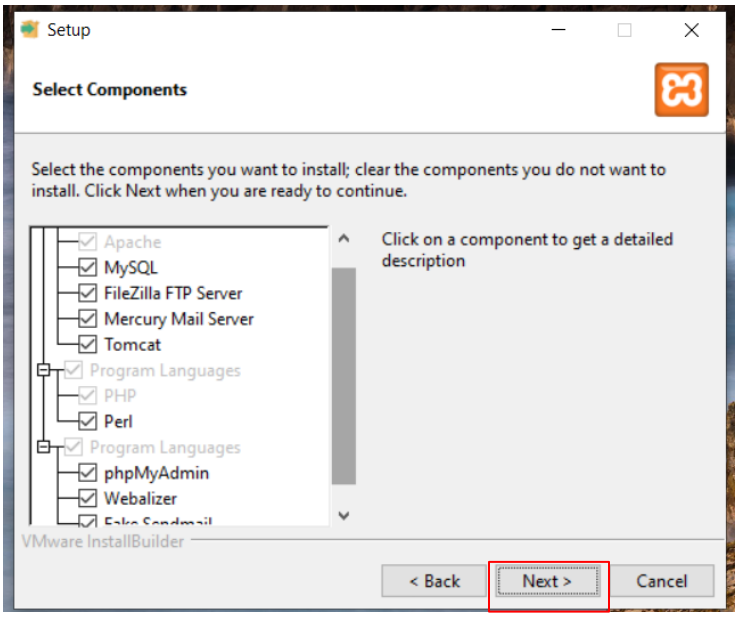


FIGURA 13 - COMPONENTES DE INSTALACIÓN XAMPP

- **Se abrirá una ventana que mostrará el directorio donde se instalará el paquete**. Si se escogió la configuración estándar, **se creará automáticamente una carpeta con el nombre "XAMPP"**.

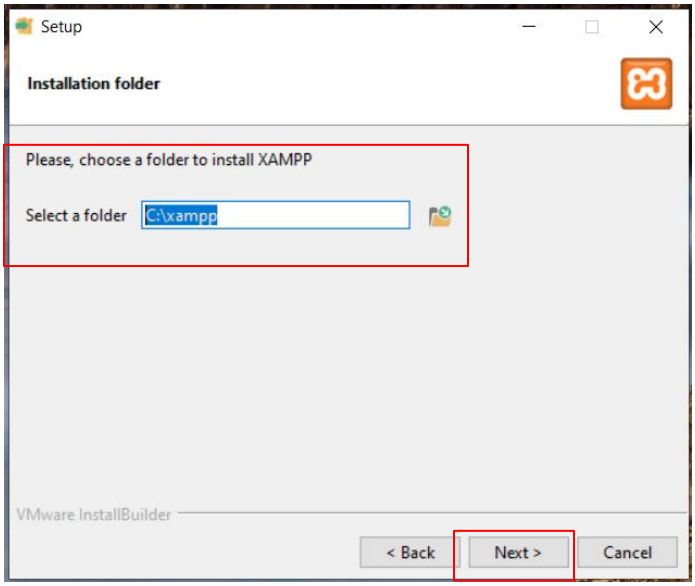


FIGURA 14 - CARPETA DE DESTINO DE INSTALACIÓN DE XAMPP

- A continuación, **se abrirá una nueva ventana que mostrará el progreso de la instalación**. Es importante tener en cuenta que **este proceso podría tomar algunos minutos**.

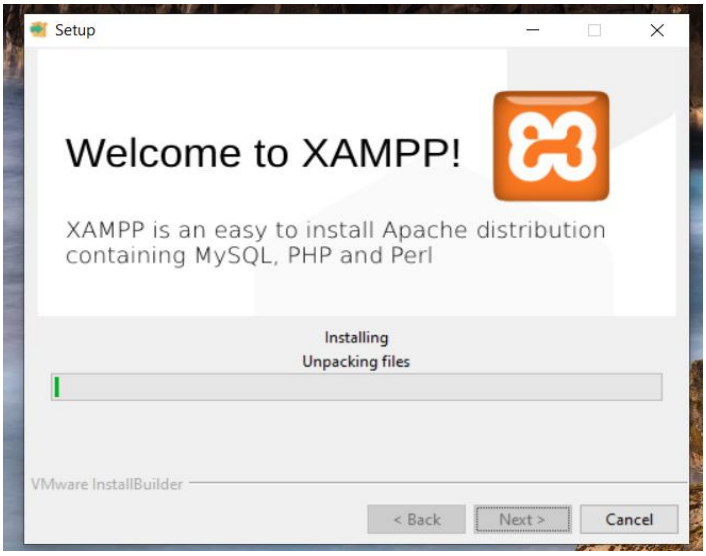


FIGURA 15 - DE INSTALACIÓN DE XAMPP

- Una vez que todos los componentes se hayan instalado, **se puede cerrar el asistente haciendo clic en "Finish"**. Para acceder inmediatamente al panel de control, **se debe marcar la casilla que pregunta si se desea hacerlo**.

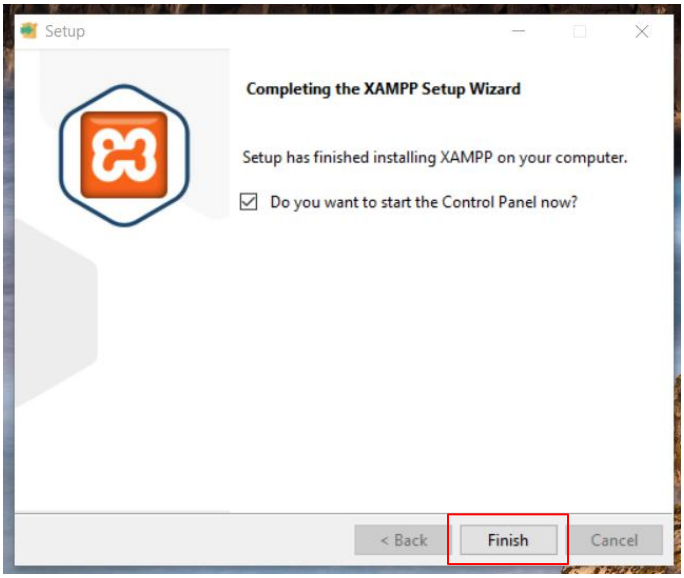


FIGURA 16 - FIN DEL PROCESO DE INSTALACIÓN DE XAMPP

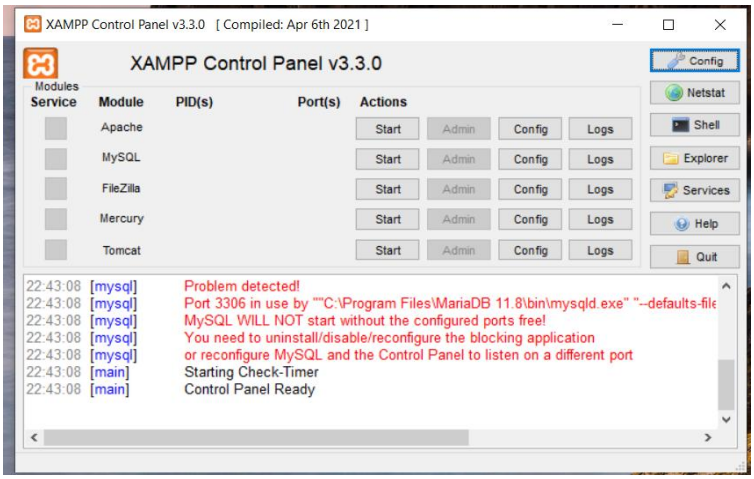


FIGURA 17 - VISTA GENERAL DE XAMPP

- Config: para configurar XAMPP, así como otros componentes aislados.
- Netstat: muestra todos los procesos en funcionamiento en el ordenador local
- Shell: lanza una ventana de comandos UNIX
- Explorer: abre la carpeta XAMPP en el explorador de Windows
- Services: muestra todos los servicios en funcionamiento
- Help: incluye enlaces a foros de usuarios
- Quit: se usar para salir del panel de control

En la parte superior del panel de control de XAMPP, **los módulos se pueden iniciar o detener individualmente** usando los comandos **"Start"** y **"Stop"** ubicados bajo la columna **"Actions"**. Los módulos que están activos se **resaltarán en verde**.

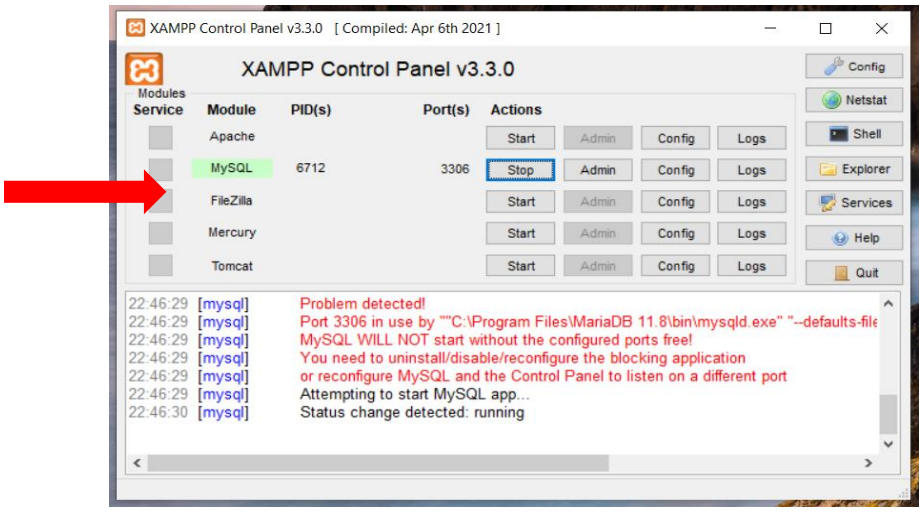


FIGURA 18 - BOTONES PARA EJECUTAR LOS DIFERENTES MÓDULOS

4.4 INSTALACIÓN DE NODE.JS

- **Se procede a descargar el archivo de instalación de Node.js** desde su sitio oficial, identificado como **"DOWNLOAD NODE.JS"**.

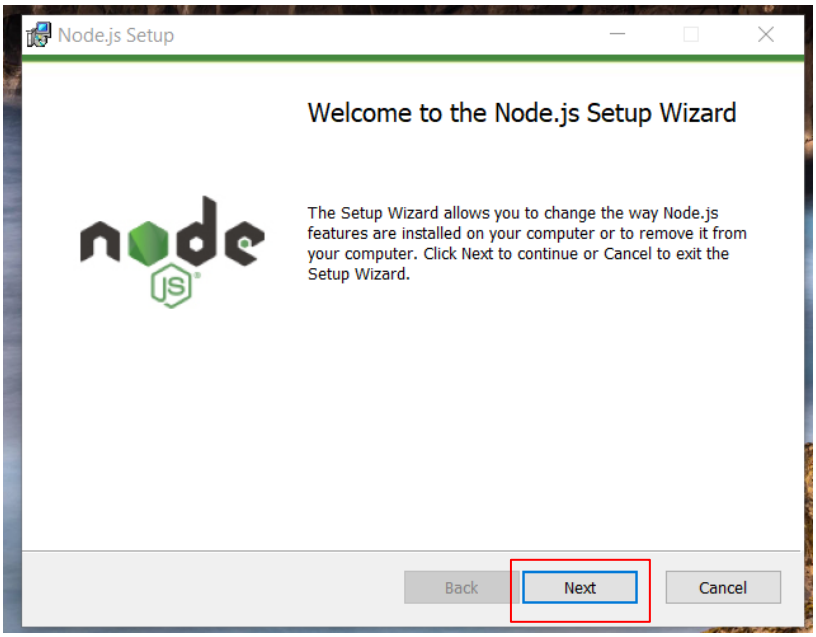


FIGURA 19 - INSTALACIÓN DE NODE.JS

- Una nueva ventana se abrirá, en la cual **se debe seleccionar la opción "Node.js runtime"** y, posteriormente, **hacer clic en "Next"** para continuar con la instalación.

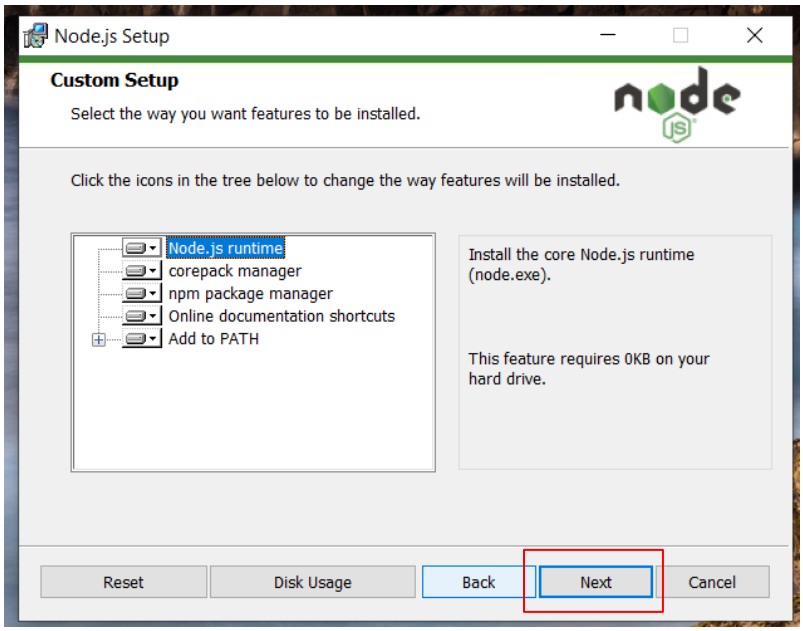


FIGURA 20 - COMPONENTE DE INSTALACIÓN DE NODE.JS

- **Se abrirá una nueva ventana**, y en ella, **se debe hacer clic en "Next"** para proceder con la instalación.

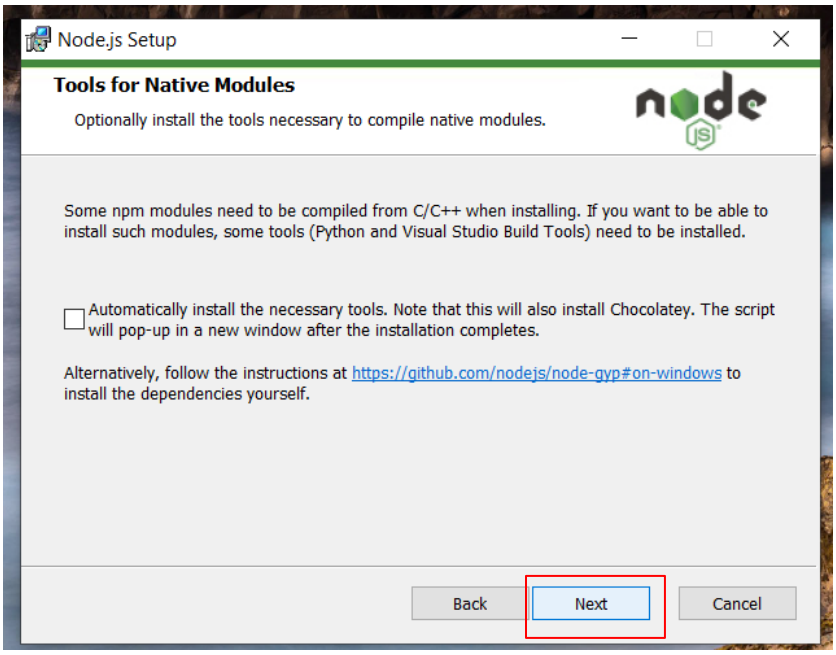


FIGURA 21 - INSTALACIÓN DE NODE.JS

- Para finalizar la instalación, **se debe hacer clic en el botón "Next"**.

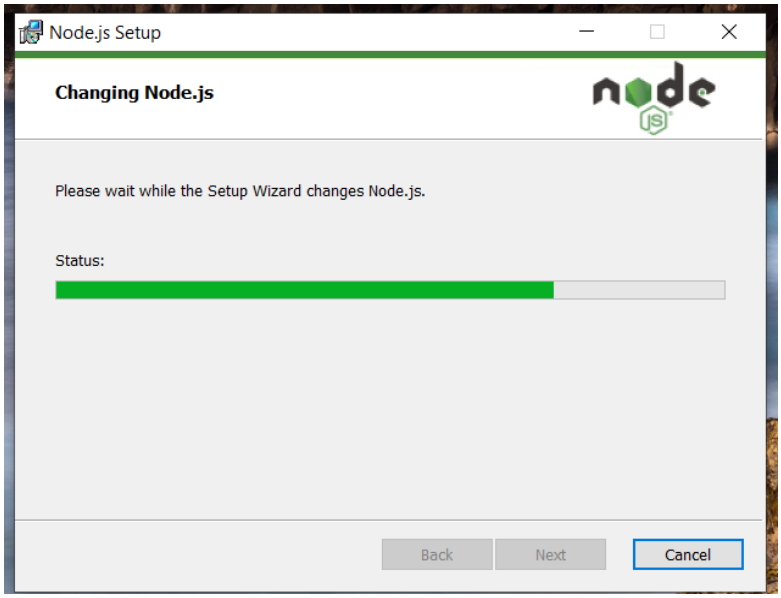


FIGURA 22 - PROCESO DE INSTALACIÓN DE NODE.JS

- Una vez que Node.js esté completamente instalado, **se puede cerrar el asistente haciendo clic en "Finish"**.

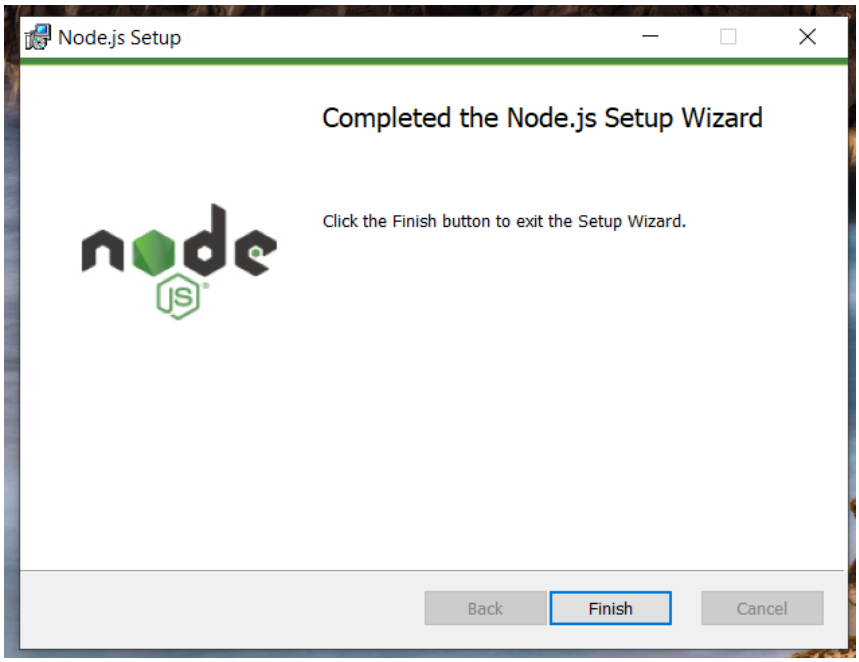


FIGURA 23 - FIN DEL INSTALACIÓN DE NODE.JS

5. INICIALIZACIÓN DE LA BASE DE DATOS

A continuación, se presenta lo que es la conformación de la base de datos del sistema donde se encuentran todas las tablas a utilizar y su breve descripción de la función que tiene. La base de datos la Inicializa por medio de "PRISMA"

No.	Tabla	Total, Registros	Descripción
1	users	13	Colección en la cual se almacenarán los datos básicos y esenciales del usuario.
2	roles	5	Colección que almacena el rol que desempeña el usuario.
3	permisos	9	Colección de las acciones específicas que pueden realizar los distintos roles en el sistema. Estos son: "crear", "leer", "actualizar" o "eliminar".
4	servidores	12	Colección que almacena toda la información relacionada a un servidor.
5	exitosas_servicios	7	Documento que almacena información sobre qué servicios tiene el servidor.
6	errores_servidor	7	Documento que almacena el Historial de Errores de un servidor.
7	grupo_servidores	5	Documento que almacena información sobre a qué grupo pertenece el servidor.
8	objeto	7	Documento que almacena todos los objetos del sistema
9	servicios	7	Colección que almacena toda la información relacionada a los servicios.
10	prisma_migrations	8	Lleva el control de las migraciones que se han aplicado.
11	errores_servicios	7	Documento que almacena el Historial de Errores de un servicio.

12	alertas_servicios	7	Documento que almacena el Historial de alertas de un servicio.
13	Bitacora	5	Documento que almacena el historial de eventos o acciones realizadas en el sistema, como accesos, modificaciones, ejecuciones de servicios y otras actividades relevantes para auditoría o trazabilidad.

6. REALIZACIÓN UN BACKUP/RESTORE

6.1 BACKUP DESDE EL SISTEMA

Para realizar una copia de seguridad desde el sistema, los usuarios deben dirigirse primero a la sección **Copia de seguridad**, ubicada en el menú.

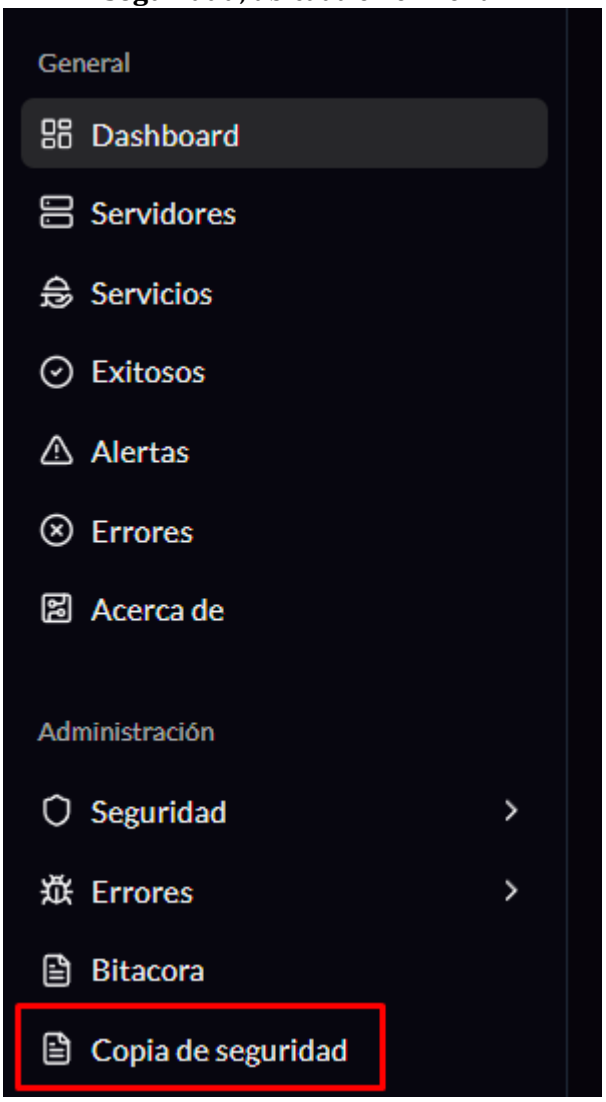


FIGURA 24 - UBICACIÓN DE LA OPCIÓN DE COPIA DE SEGURIDAD.

Una vez en la pantalla correspondiente, los usuarios deben hacer clic en **Crear Backup**, lo cual generará la copia de seguridad.

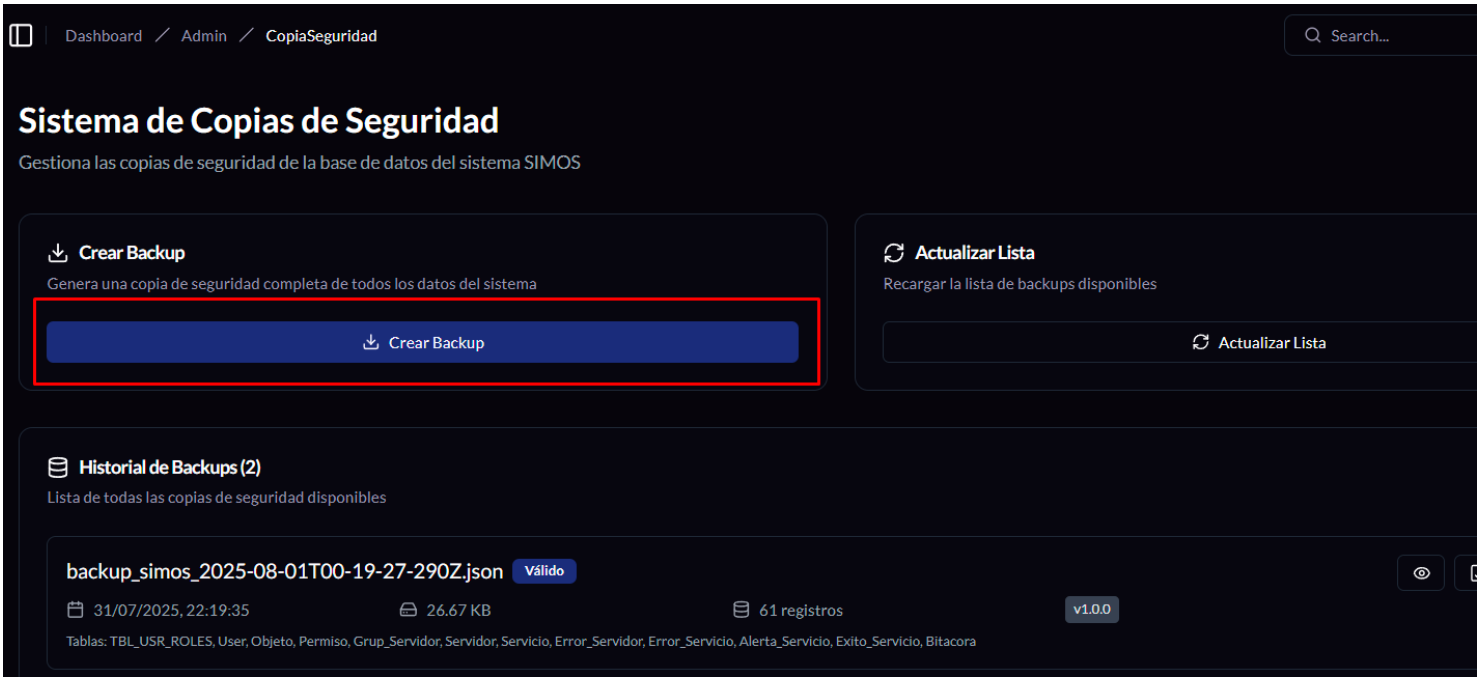


FIGURA 25 - CREACIÓN DE COPIA DE SEGURIDAD.

Una vez creado el backup, la pantalla mostrará los datos de la creación, como la fecha y la cantidad de registros guardados.

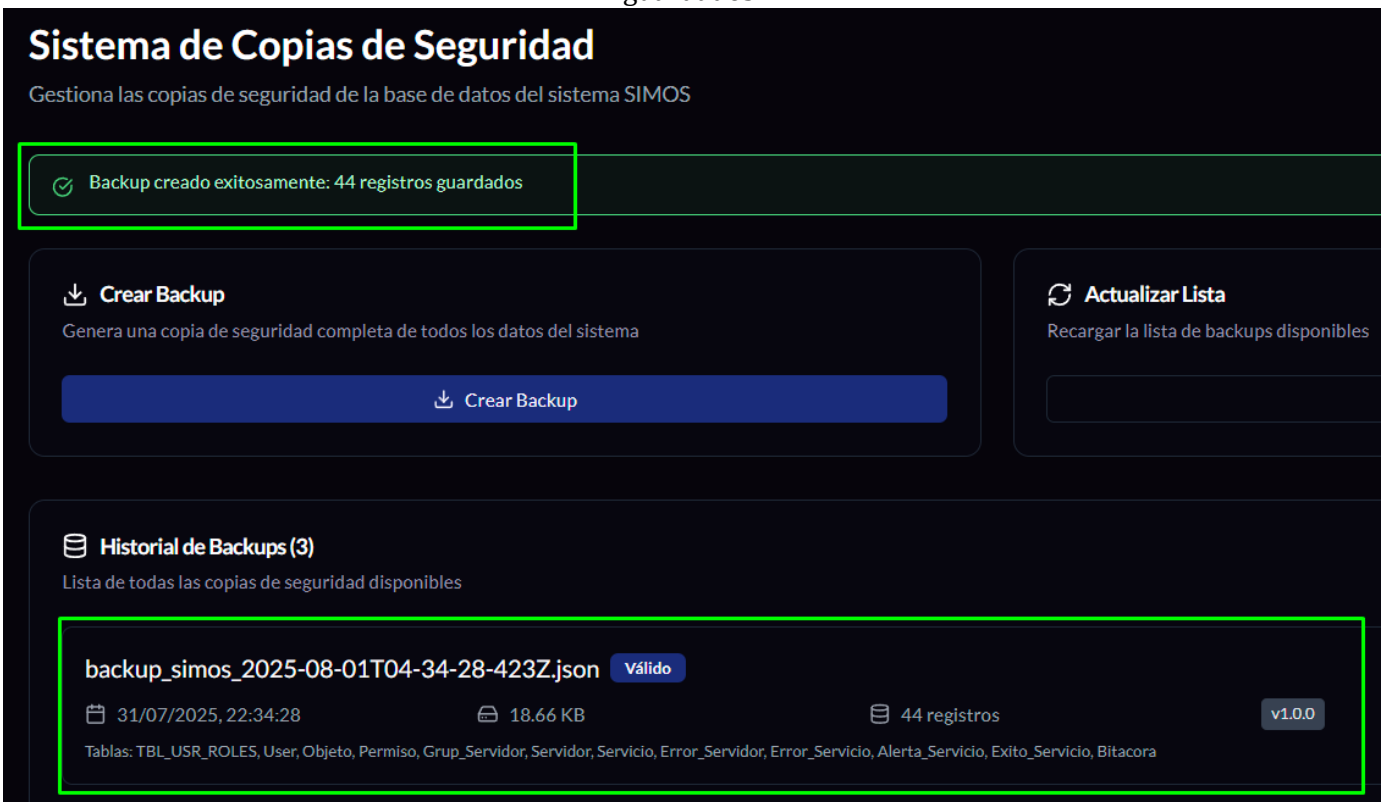


FIGURA 26 - BACKUP REALIZADO.

6.2 BACKUP DESDE LA BASE DE DATOS

Para realizar el backup desde la base de datos, los usuarios deben acceder a PostgreSQL, seleccionar el servidor y hacer clic derecho sobre él para desplegar un menú de opciones.

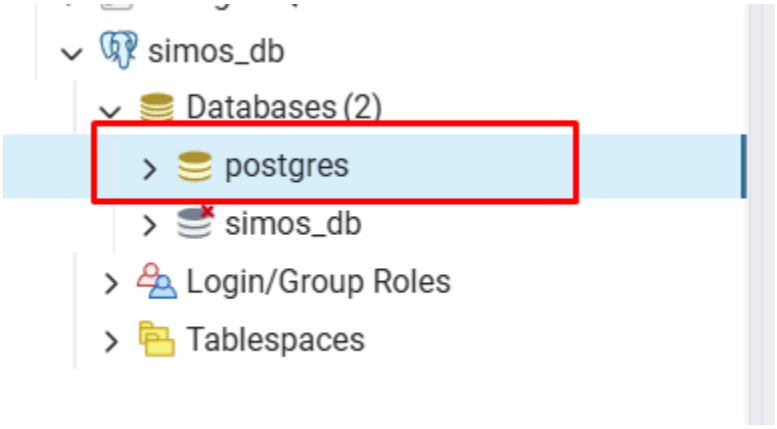


FIGURA 27 - VISTA DEL SERVIDOR.

En el menú desplegado, los usuarios deben hacer clic en **Backup.**

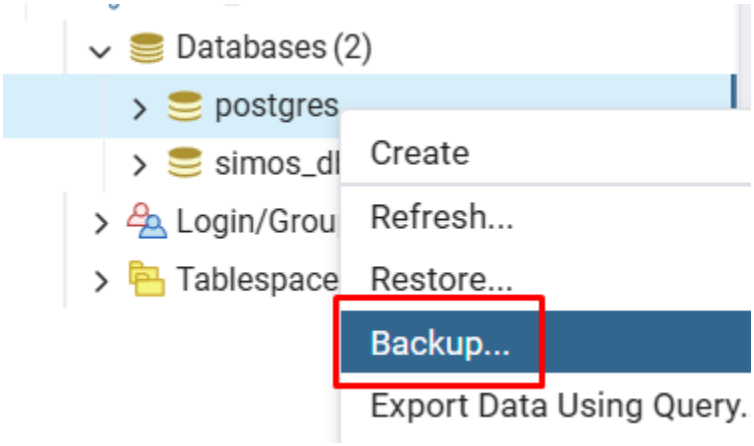


FIGURA 28 - MENÚ DE OPCIONES.

En esta ventana, los usuarios deben completar los campos correspondientes para generar el backup y seleccionar la ruta en la que desean guardarlo.

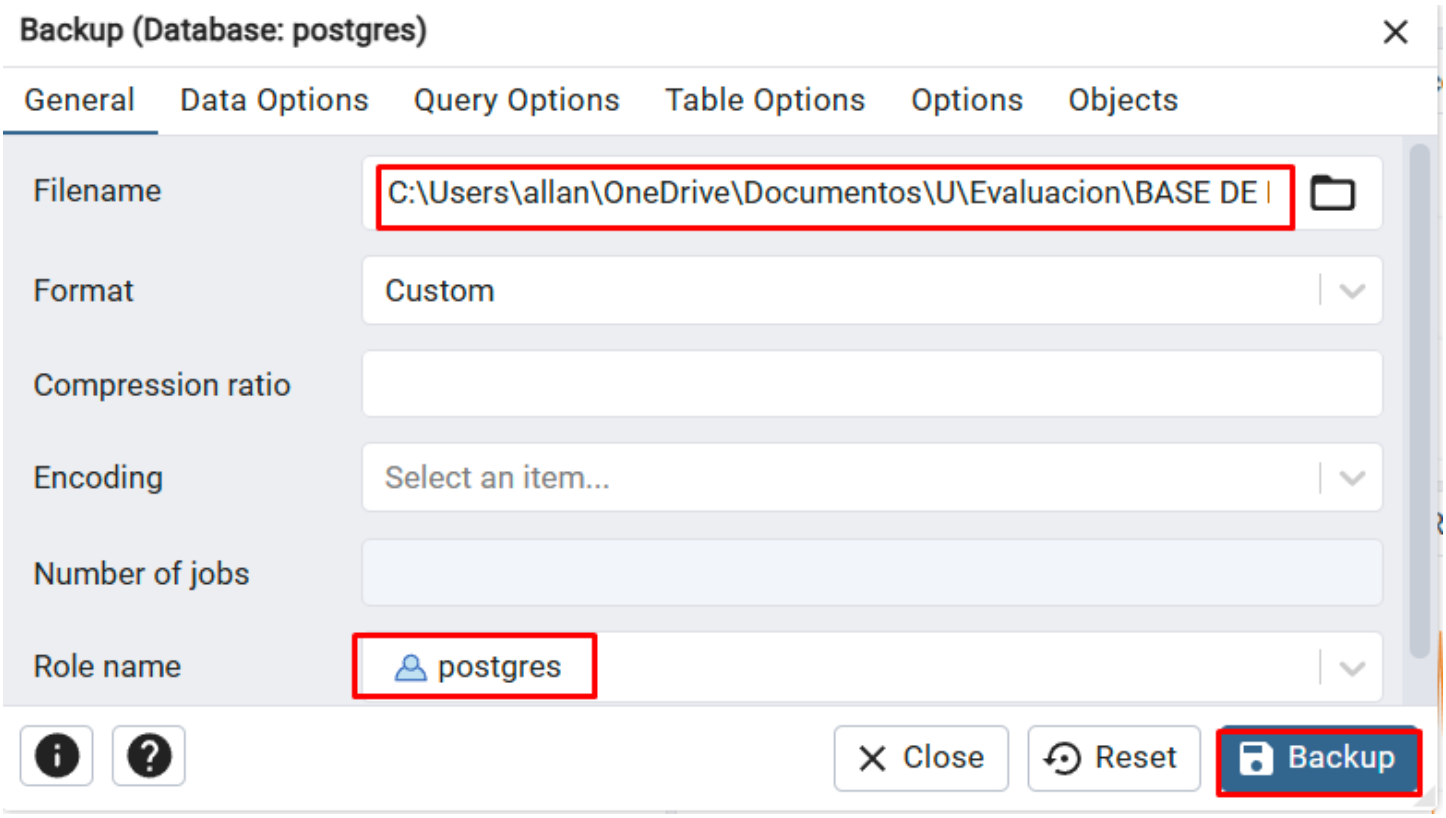


FIGURA 29 - CAMPOS A LLENAR.

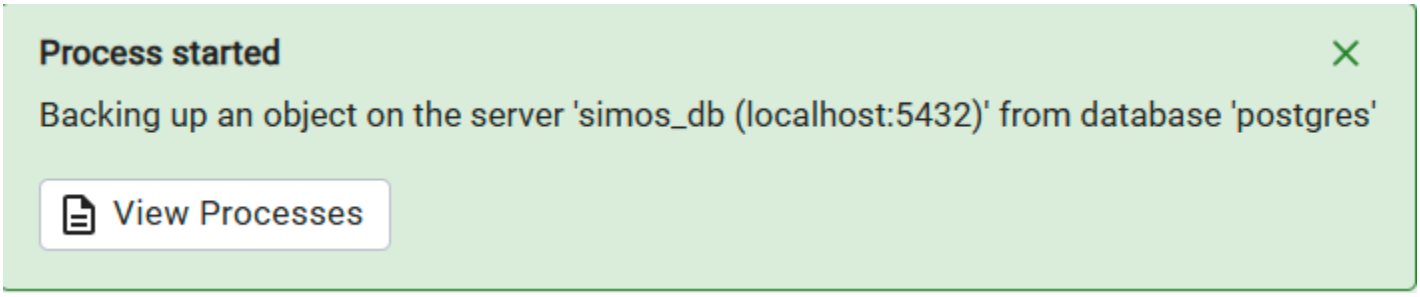


FIGURA 30 - BACKUP DE BASE DE DATOS CREADO FORMA MANUAL.

6.3 RESTORE DESDE EL SISTEMA

Para restaurar el backup creado anteriormente, los usuarios deben hacer clic sobre este ícono.



FIGURA 31 - PRIMER PASO DE RESTAURACIÓN.

Se presentará una pantalla para confirmar si se desea realizar la restauración, ya que esta acción borrará todos los cambios realizados desde la creación del backup en adelante.

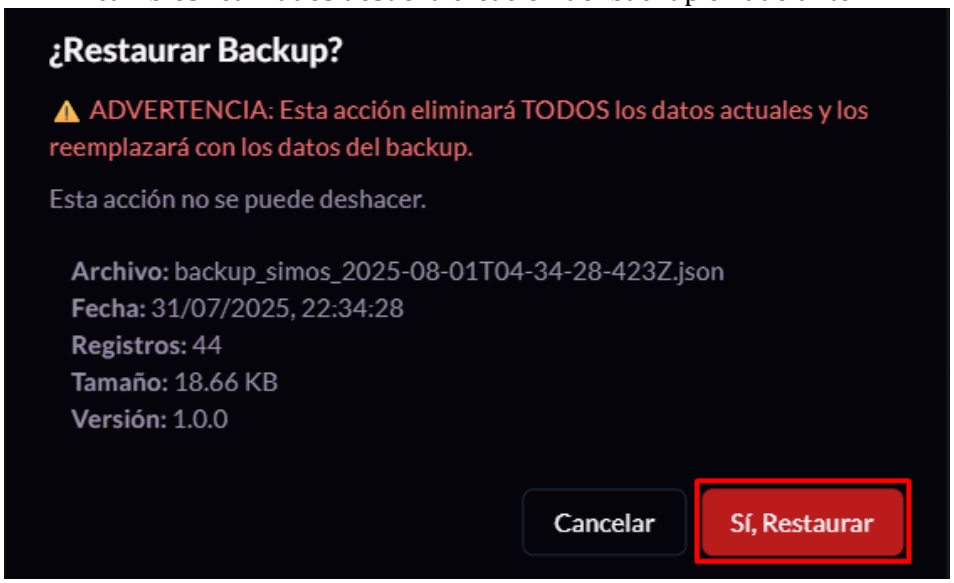


FIGURA 32 - CONFIRMACIÓN DE RESTAURACIÓN.

Una vez finalizada la restauración, se mostrará en pantalla una confirmación que indica que el proceso se realizó con éxito, junto con la cantidad de registros restaurados.



FIGURA 33 - RESTAURACIÓN FINALIZADA.

6.4 RESTORE DESDE LA BASE DE DATOS

El proceso inicia seleccionando la base de datos donde se desea realizar la restauración, haciendo clic derecho sobre ella.



FIGURA 34 - BASE DE DATOS DONDE SE RESTAURARÁ.

Se nos desplegará un menú y seleccionaremos **restore**.

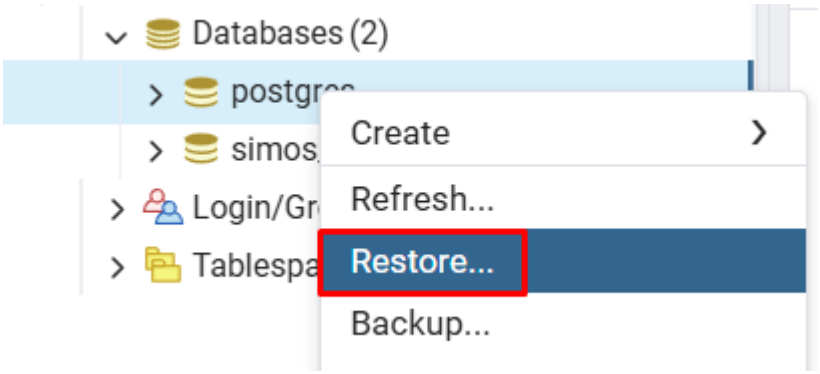


FIGURA 35 - OPCIÓN DE RESTAURAR.

Una vez en la pantalla de restauración, los usuarios deben hacer clic en la carpeta ubicada al lado derecho del campo **Filename**, lo que los dirigirá a la carpeta donde se guardó el backup. Allí, deben cambiar el formato de los archivos mostrados a **SQL File (.sql)**, lo que permitirá visualizar los archivos disponibles para restaurar. Finalmente, se selecciona el archivo deseado y se hace clic en **Abrir**.

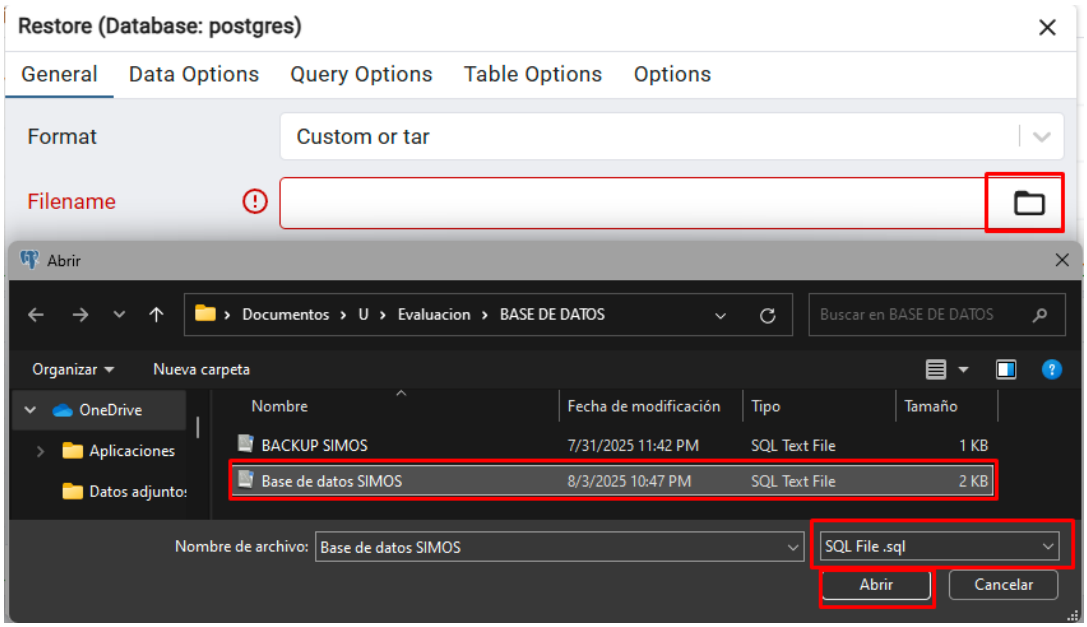


FIGURA 36 - SELECCIÓN DE ARCHIVO BACKUP.

Se cambia el rol a **Postgres** y se hace clic en **Restore**.

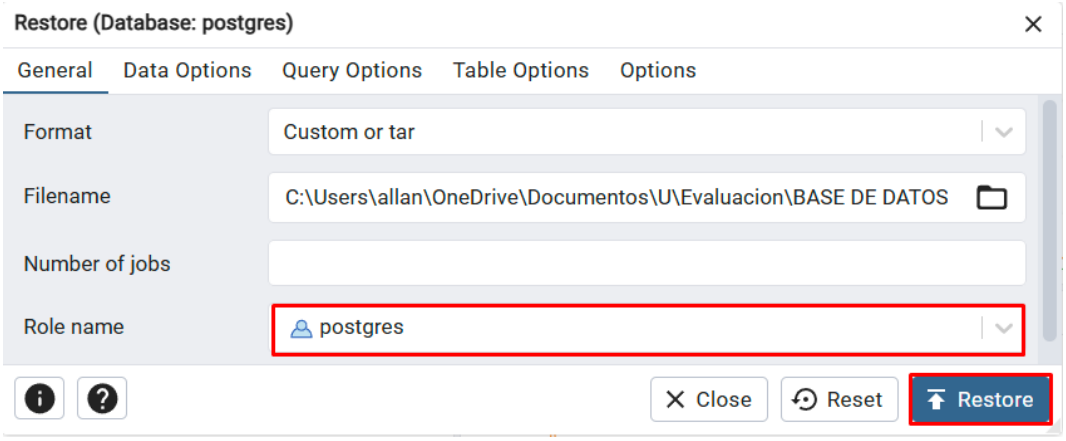


FIGURA 37 - ÚLTIMO PASO PARA LA RESTAURACIÓN.

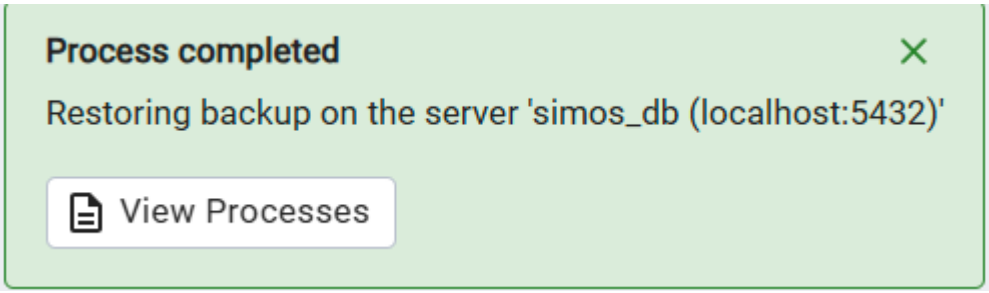


FIGURA 38 - RESTAURACIÓN TERMINADA.

7. SEGURIDAD, PARAMETROS, USO DE ROLES

7.1 PARÁMETROS

Como parte de los parámetros que se entregan con el sistema, se incluyen los módulos funcionales y la matriz de acceso por roles. Para los usuarios generales, el sistema proporciona acceso a los módulos de *Dashboard*, *Servidores*, *Servicios*, *Exitosos*, *Alertas*, *Errores* y *Acerca de*.

Por su parte, los administradores tienen acceso a los módulos anteriores, además de funcionalidades avanzadas para la gestión del sistema, como el manejo de *Usuarios*, *Pantallas*, *Roles*, *Permisos*, *Errores* del sistema y la *Bitácora* de actividades.

La matriz de acceso define los permisos iniciales (Insertar, Editar, Eliminar y Consultar) según el rol asignado, asegurando un control estructurado sobre las acciones que cada tipo de usuario puede realizar dentro del sistema.

7.2 MANTENIMIENTO DE ROLES

7.2.1 Rol de Administrador:

El administrador tiene acceso completo a todas las funciones del sistema. Como su nombre lo indica el administrador con el cual se podrán hacer los mantenimientos necesarios del sistema, claro está, que debido a sus privilegios solo un encargado deberá hacer uso de este usuario.

El administrador puede:

- Consultar, Insertar, actualizar y eliminar.
- Asignar roles (por ejemplo: usuario).
- Gestionar permisos específicos por usuario.
- Monitorea el funcionamiento de los servidores.
- Reacciona ante alertas y fallas detectadas por el sistema.

7.2.2 Rol de Usuario:

Este rol está diseñado para usuarios con acceso limitado, restringido a funciones operativas básicas del sistema cómo visualizar información de (Dashboard, Servidores, Servicios, Exitosos, Alertas, Errores en servicios o servidores, Acerca de), pero no tienen permisos para modificar configuraciones, gestionar otros usuarios o acceder a secciones administrativas como módulo de seguridad.

Cantidad: Habrá múltiples usuarios con este rol, lo que permite escalar el número de usuarios según las necesidades del sistema sin comprometer la seguridad o el control esperado.

Los Usuarios Normales son los operadores del sistema, quienes interactúan con las funcionalidades para realizar tareas específicas, pero sin capacidad de alterar configuraciones críticas o acceder a datos sensibles más allá de lo permitido.

7.3 PERMISOS DEL SISTEMA

Por política de seguridad interna de Grupo Farinter, estos permisos se restringen exclusivamente al rol de administrador, para reducir el riesgo de errores operativos, accesos indebidos o manipulaciones no autorizadas del sistema de monitoreo.

- Crear, editar y eliminar usuarios.
- Asignar o revocar roles.
- Restablecer contraseñas.
- Bloquear o desbloquear cuentas.
- Control de accesos
- Añadir o eliminar servicios, servidores, etc.
- Configurar políticas de seguridad (contraseñas, sesiones, autenticación)

7.4 MATRIZ DE ACCESO AL SISTEMA

La presente matriz define los niveles de acceso que poseen los distintos roles dentro del sistema sobre cada uno de los objetos o módulos disponibles. En ella se especifican los permisos otorgados para insertar, editar, eliminar y consultar información, permitiendo así un control claro y estructurado de las acciones que cada rol puede ejecutar en el sistema.

No.	Rol	Objeto	Insertar	Editar	Eliminar	Consultar
1	Admin	Todas las pantallas	Y	Y	Y	Y
2	User	Acerca De	N	N	N	Y
3	User	Dashboard	N	N	N	Y
4	User	Errores	N	N	N	Y
5	User	Servicios	N	N	N	Y
6.	User	Alertas	N	N	N	Y

8. ANEXOS

8.1 INSTALACIÓN DE APLICATIVO EN SERVIDOR PRIVADO VIRTUAL (VPS)

Para ingresar al Servidor Privado Virtual (VPS), se debe abrir una ventana de CMD e ingresar el comando ssh root@168.231.66.51. Una vez dentro, se utiliza pm2 status para verificar si la aplicación está activa. Posteriormente, se ejecuta pm2 log para visualizar los registros (logs) de la aplicación.

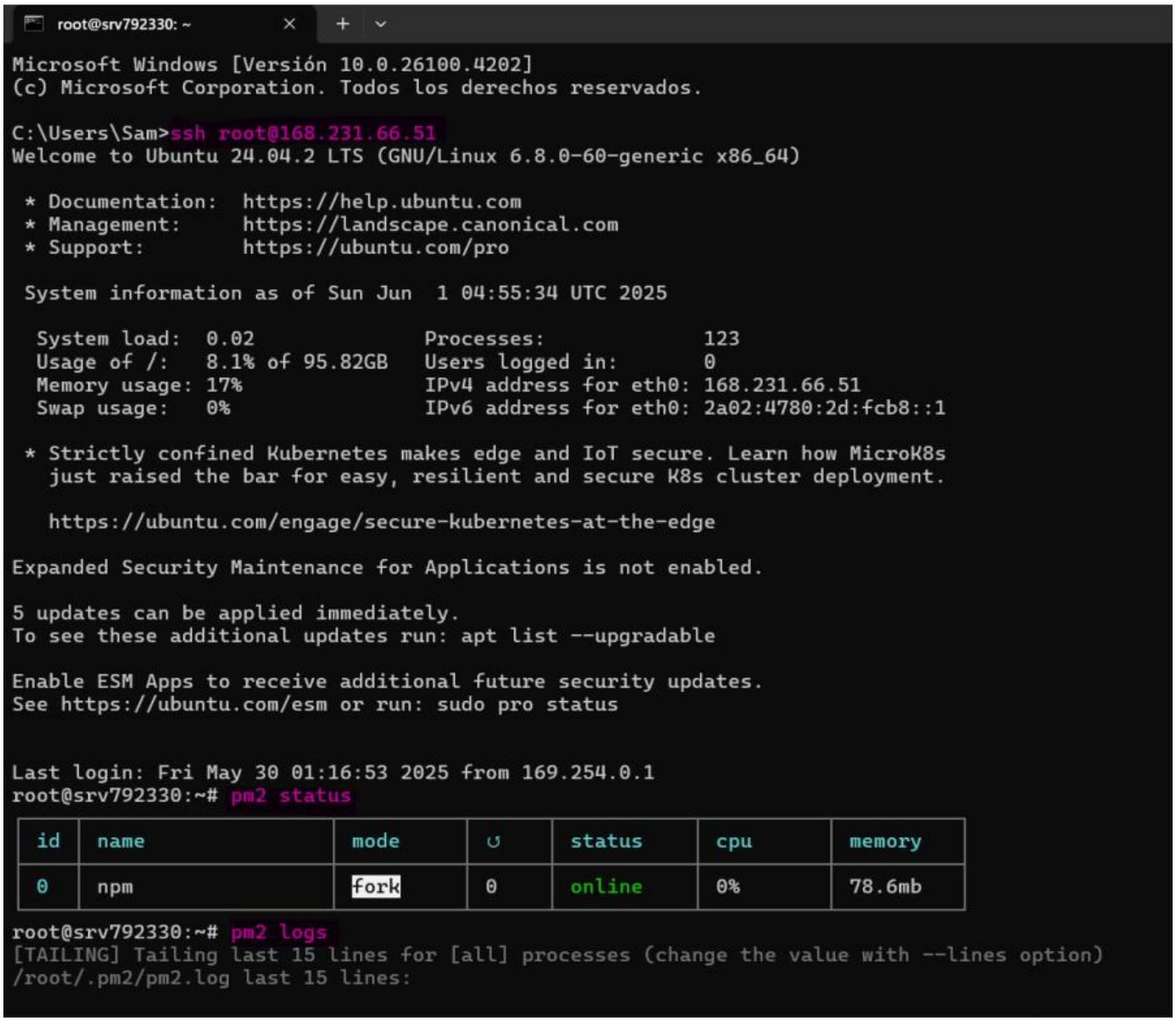


FIGURA 38 - INGRESO AL SERVIDOR PRIVADO VIRTUAL (VPS)

El error que **devuelve la base de datos** es el siguiente:

```
0|npm  | }
0|npm  | {
0|npm  |   err: PrismaClientKnownRequestError:
0|npm  |   Invalid `prisma.user.findUnique()` invocation:
0|npm  |
0|npm  |
0|npm  |   Database `simos_db` does not exist on the database server at `localhost:3306`.
0|npm  |       at zn.handleRequestError (/root/simos/node_modules/@prisma/client/runtime/library.js:121:7459)
0|npm  |       at zn.handleAndLogRequestError (/root/simos/node_modules/@prisma/client/runtime/library.js:121:6784)
0|npm  |       at zn.request (/root/simos/node_modules/@prisma/client/runtime/library.js:121:6491)
0|npm  |       at async l (/root/simos/node_modules/@prisma/client/runtime/library.js:130:9778)
0|npm  |       at async Object.authorize (/root/simos/.next/server/chunks/805.js:1:6491)
0|npm  |       at async aj (/root/simos/.next/server/chunks/853.js:417:42659)
0|npm  |       at async aB (/root/simos/.next/server/chunks/853.js:417:52202)
0|npm  |       at async aV (/root/simos/.next/server/chunks/853.js:417:56463)
0|npm  |       at async a2 (/root/simos/.next/server/chunks/853.js:417:61759)
0|npm  |       at async o (/root/simos/.next/server/app/auth/login/page.js:1:10926) {
0|npm  |     code: 'P1003',
0|npm  |     meta: {
0|npm  |       modelName: 'User',
0|npm  |       database_name: 'simos_db',
0|npm  |       database_host: 'localhost',
0|npm  |       database_port: 3306
0|npm  |     },
0|npm  |     clientVersion: '6.6.0'
0|npm  |   },
0|npm  |   provider: 'credentials'
0|npm  | }
```

FIGURA 39 - ERROR DEVUELTO BASE DE DATOS

El error sugiere que la base de datos no existe o ha dejado de existir. Para verificar si la base de datos está activa, se utiliza el comando docker ps.

```
root@srv792330:~# docker ps
CONTAINER ID   IMAGE     COMMAND                  CREATED      STATUS      PORTS                                                  NAMES
538d80f35ec6   mysql     "docker-entrypoint.s…"   7 days ago   Up 2 days   0.0.0.0:3306->3306/tcp, [::]:3306->3306/tcp, 33060/tcp   simos-db-1
root@srv792330:~#
```

FIGURA 40 - VERIFICACIÓN DE BASE DE DATOS ACTIVA

Como se observa en la imagen previa, la base de datos sí existe. A pesar de esto, incluso al detener la base de datos con docker stop utilizando el nombre del contenedor, el sistema continúa mostrando el error de que la base de datos no existe.

```
For more help on how to use Docker, head to https://docs.docker.com/go/guides/
root@srv792330:~# docker stop 538d80f35ec6
538d80f35ec6
root@srv792330:~# docker ps -a
CONTAINER ID   IMAGE     COMMAND                  CREATED      STATUS                      PORTS     NAMES
538d80f35ec6   mysql     "docker-entrypoint.s…"   7 days ago   Exited (0) 3 seconds ago              simos-db-1
root@srv792330:~# docker start 538d80f35ec6
538d80f35ec6
root@srv792330:~# docker ps -a
CONTAINER ID   IMAGE     COMMAND                  CREATED      STATUS          PORTS                                                  NAMES
538d80f35ec6   mysql     "docker-entrypoint.s…"   7 days ago   Up 25 seconds   0.0.0.0:3306->3306/tcp, [::]:3306->3306/tcp, 33060/tcp   simos-db-1
```

FIGURA 41 - VERIFICACIÓN DE BASE DE DATOS EXISTENTE

Y el **error** se mantiene.

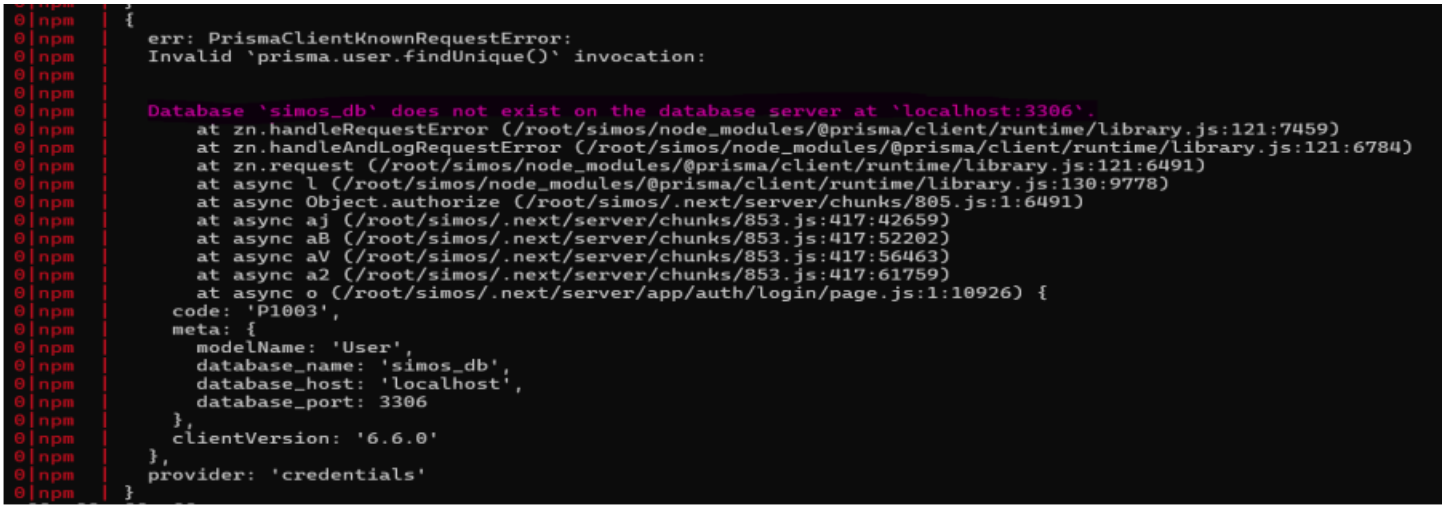


FIGURA 42 - ERROR SE MANTIENE

8.2 PROCESO DE LEVANTAMIENTO VPS

Primeramente, **se debe importar el proyecto de GitHub en el directorio raíz** utilizando el comando:

git clone https://github.com/innovasoftDev/simos.git

Posteriormente, **se valida que la carpeta haya sido creada** ejecutando el comando ls.Posteriormente, **se valida que la carpeta haya sido creada** ejecutando el comando ls.

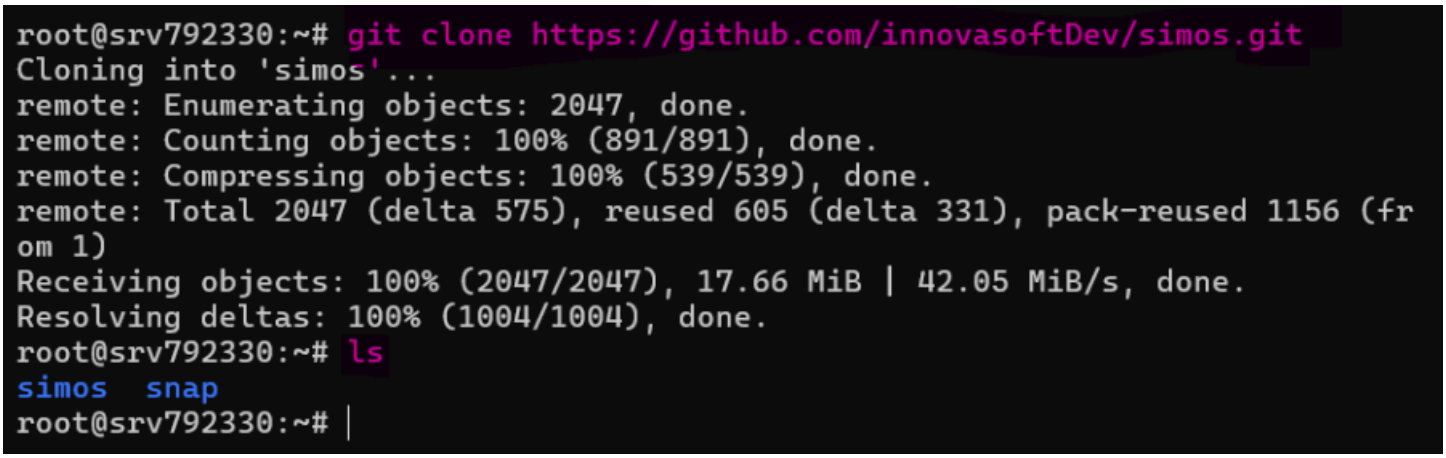


FIGURA 43 - IMPORTACIÓN DE PROYECTO

Se debe navegar a la carpeta simos que fue creada por el comando anterior, utilizando el comando: **cd simos.** Una vez dentro de la carpeta, **se ejecutará el comando** para generar una clave: **openssl rand -base64 32.**

Este comando generará un código que deberá ser copiado. Posteriormente, se ejecutará el comando nano .env.template para editar el archivo.

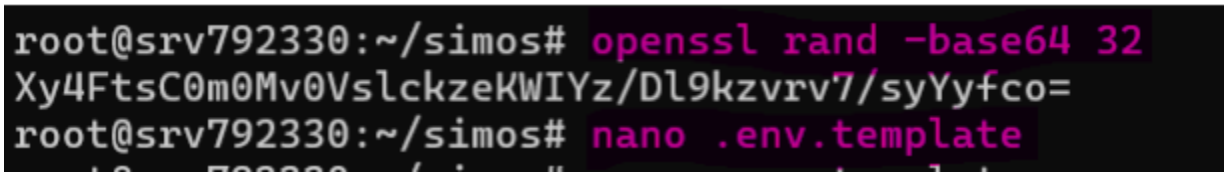


FIGURA 44 - NAVEGACIÓN EN LA CARPETA SIMOS

Al abrir el archivo .env.template, se debe localizar la variable AUTH_SECRET. Dentro de las comillas correspondientes a esta variable, se procederá a pegar el código que se generó previamente.

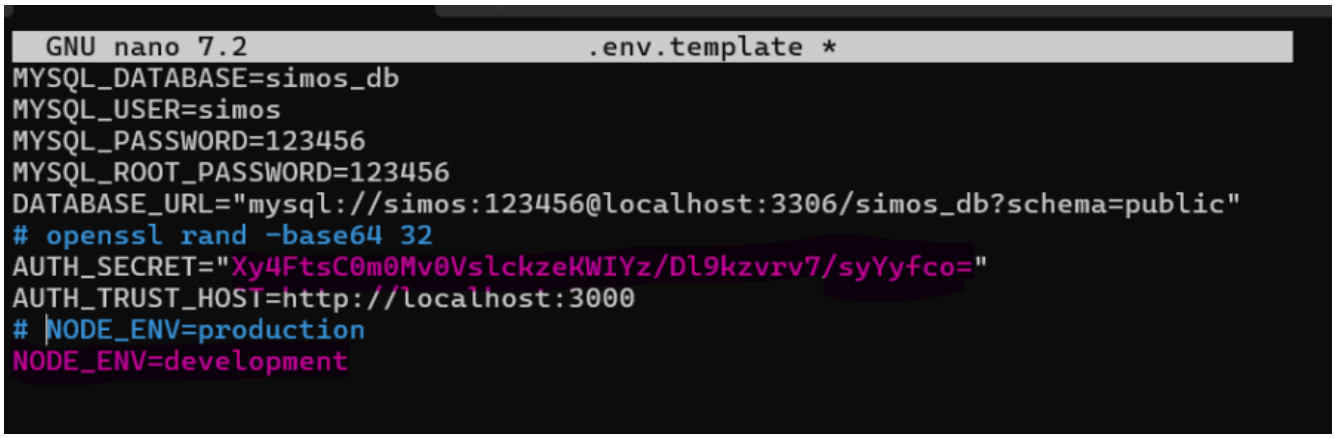


FIGURA 45 - LOCALIZACIÓN DE VARIABLE AUTH_SECRET

Además, se debe comentar la línea NODE_ENV=production para deshabilitar, y se deja activa la línea NODE_ENV=development. Esto permite realizar cambios en el proyecto antes de su despliegue final.

Una vez hechas las modificaciones, se guardan los cambios presionando CTRL+X.

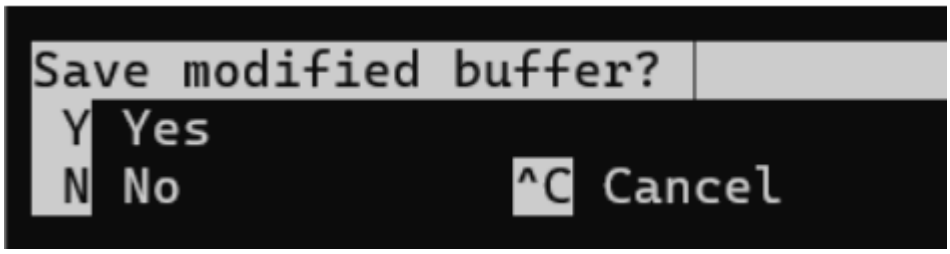


FIGURA 46 - CONFIRMACIÓN

Se debe teclear "Y" para confirmar el guardado. Luego, el sistema preguntará si se desea cambiar el nombre del archivo; se debe responder "Y" nuevamente, seguido del nuevo nombre: .env.

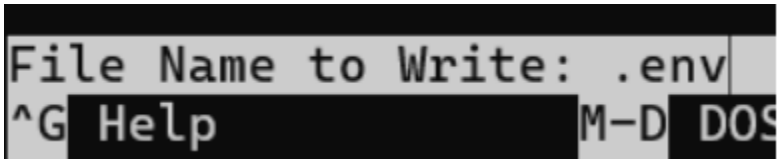


FIGURA 47 - CONFIRMACIÓN FINAL

Finalmente, **se debe presionar Enter** para confirmar.

Ahora, se ejecuta el comando npm install para instalar las dependencias del proyecto. Una vez finalizada la instalación de estas, se procede a ejecutar docker compose up -d. Este comando creará la base de datos en Docker.

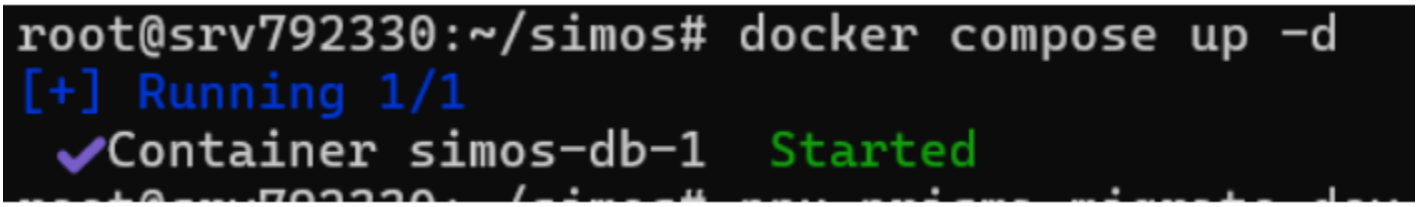


FIGURA 48 - EJECUCIÓN DE COMANDO NPM INSTALL

Para establecer la estructura de tablas necesaria, se ejecuta el comando npx prisma db push. Seguidamente, se ejecuta npm run seed. Este último comando ingresa en las tablas la información base requerida para el correcto funcionamiento del proyecto.

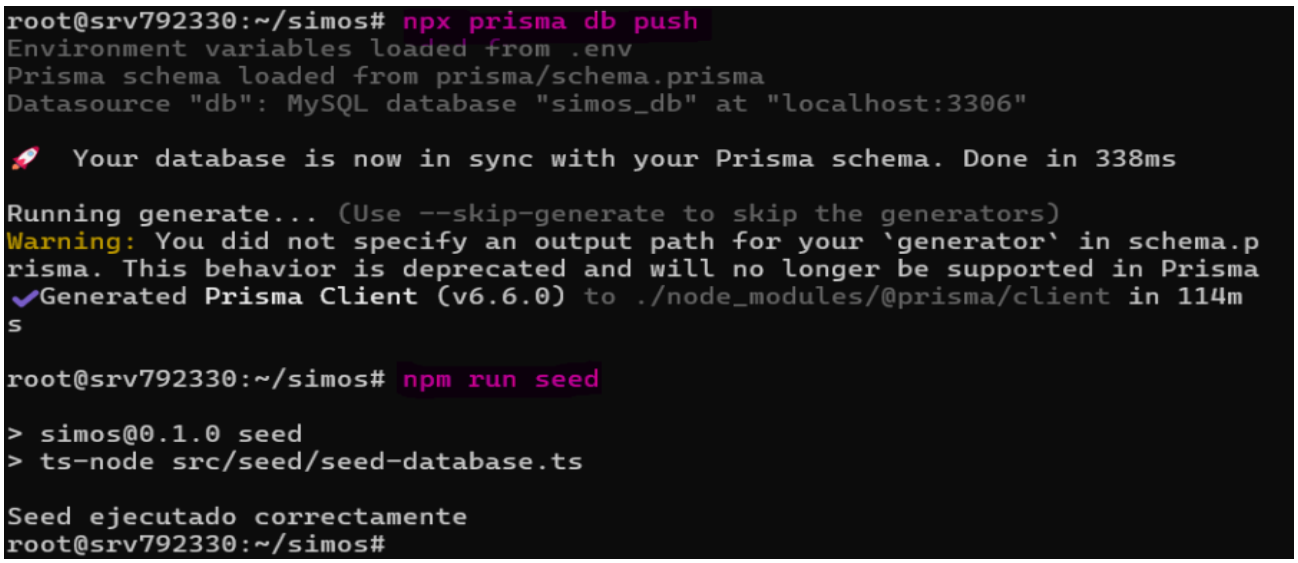


FIGURA 49 - LEVANTAMIENTO DE BASE DE DATOS

Ahora, se procede a modificar nuevamente el archivo .env utilizando el comando nano .env.

Dentro de este archivo, se debe descomentar la línea NODE_ENV=production y, al mismo tiempo, se comenta NODE_ENV=development. Esto asegura que el proyecto funcione en un entorno de producción.

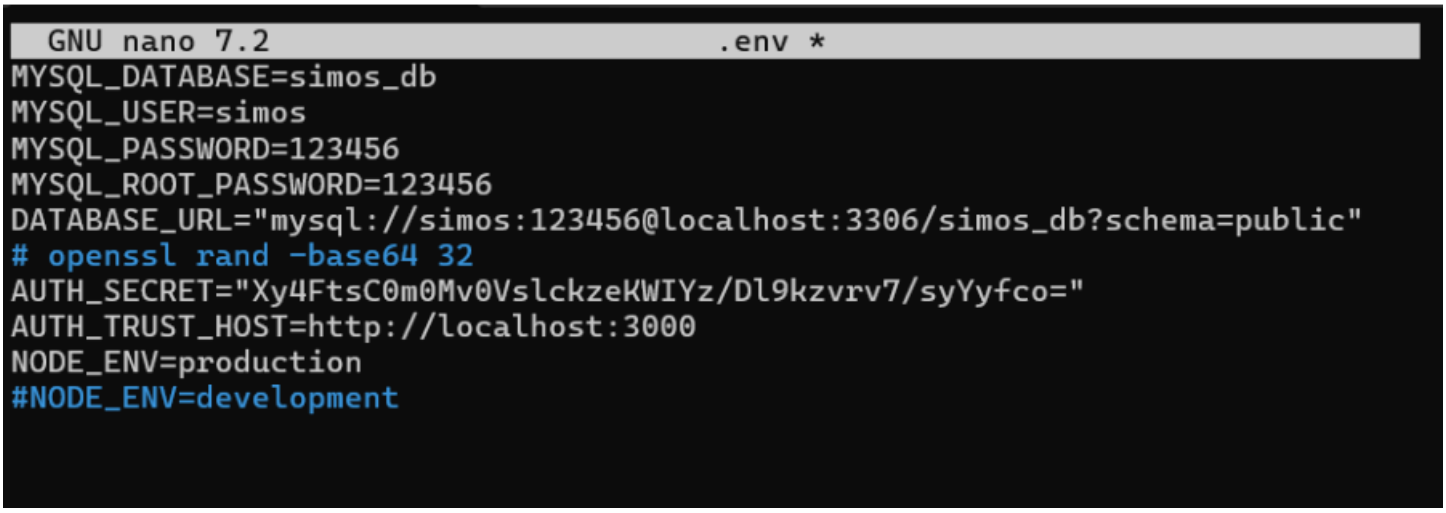


FIGURA 50 - MODIFICACIÓN DE ARCHIVO .ENV

Finalmente, se ejecuta el comando npm run build para crear el compilado optimizado para producción.

```
 ✓ Compiled successfully
 ✓ Linting and checking validity of types
 ✓ Collecting page data
 ✓ Generating static pages (25/25)
 ✓ Collecting build traces
 ✓ Finalizing page optimization

Route (app)                                 Size     First Load JS
┌ ƒ /                                       165 B          87.6 kB
├ ƒ /_not-found                             165 B          87.6 kB
├ ƒ /401                                    1.24 kB        96.8 kB
├ ƒ /403                                    1.22 kB        96.8 kB
├ ƒ /404                                    1.24 kB        96.8 kB
├ ƒ /503                                    1.25 kB        96.9 kB
├ ƒ /api/auth/[...nextauth]                 0 B                0 B
├ ƒ /auth/forgot-password                   2.6 kB          129 kB
├ ƒ /auth/login                             4.21 kB         135 kB
├ ƒ /auth/new-account                       6.41 kB         136 kB
├ ƒ /dashboard                              165 B          87.6 kB
├ ƒ /dashboard/about                        165 B          87.6 kB
├ ƒ /dashboard/admin                        165 B          87.6 kB
├ ƒ /dashboard/admin/pantallas              7.92 kB         209 kB
├ ƒ /dashboard/admin/permisos               7.88 kB         203 kB
├ ƒ /dashboard/admin/roles                  5.74 kB         192 kB
├ ƒ /dashboard/admin/users                  9.45 kB         210 kB
├ ƒ /dashboard/alertas                      5.86 kB         183 kB
├ ƒ /dashboard/errores                      5.86 kB         183 kB
├ ƒ /dashboard/exitosos                     5.86 kB         183 kB
├ ƒ /dashboard/overview                     131 kB          257 kB
├ ƒ /dashboard/profile                      300 B          92.9 kB
├ ƒ /dashboard/servicios                    7.87 kB         209 kB
└ ƒ /dashboard/servidores                   8.08 kB         209 kB
+ First Load JS shared by all              87.4 kB
  ├ chunks/2117-e827d76c2e9fcf64.js         31.8 kB
  ├ chunks/fd9d1056-cebc8f72702f1cbb.js     53.6 kB
  └ other shared chunks (total)             2.01 kB


ƒ  (Dynamic)  server-rendered on demand

root@srv792330:~/simos#
```

FIGURA 51 - EJECUCIÓN DE COMANDO NPM RUN BUILD

Una vez creado el compilado, se procede a levantar el servicio en producción utilizando el comando pm2 start npm -- start.

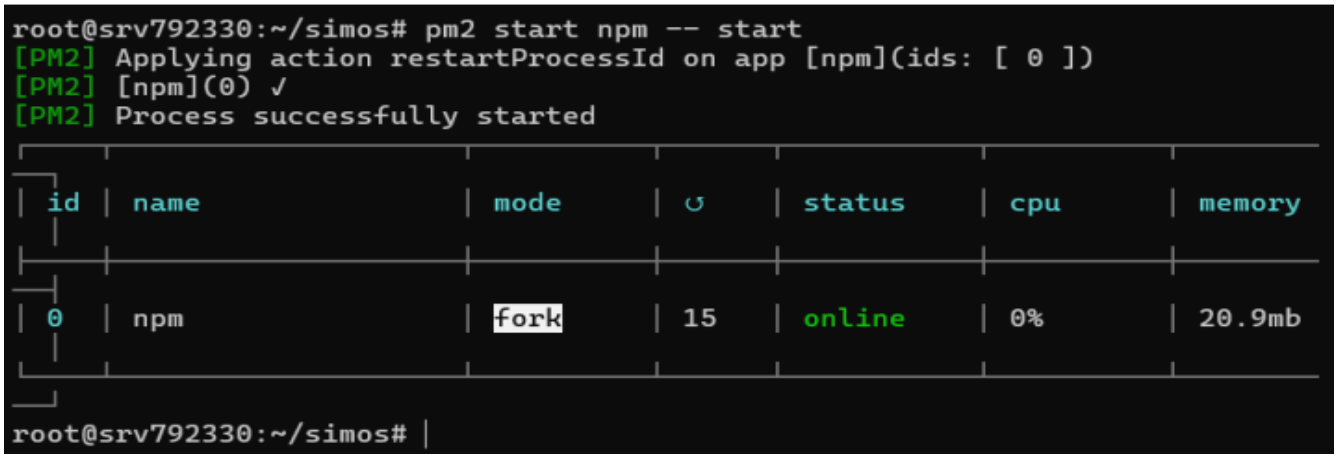


FIGURA 52 - LEVANTAMIENTO DEL SERVICIO

Con los pasos anteriores, **la aplicación ya debería estar levantada**. Para validar su correcto funcionamiento, **se debe acceder a la siguiente ruta en el navegador**:

https://www.innovasoftdev.online/

Una vez en la página, **se ingresa con las credenciales de administrador**, y con eso, la configuración estará completa.

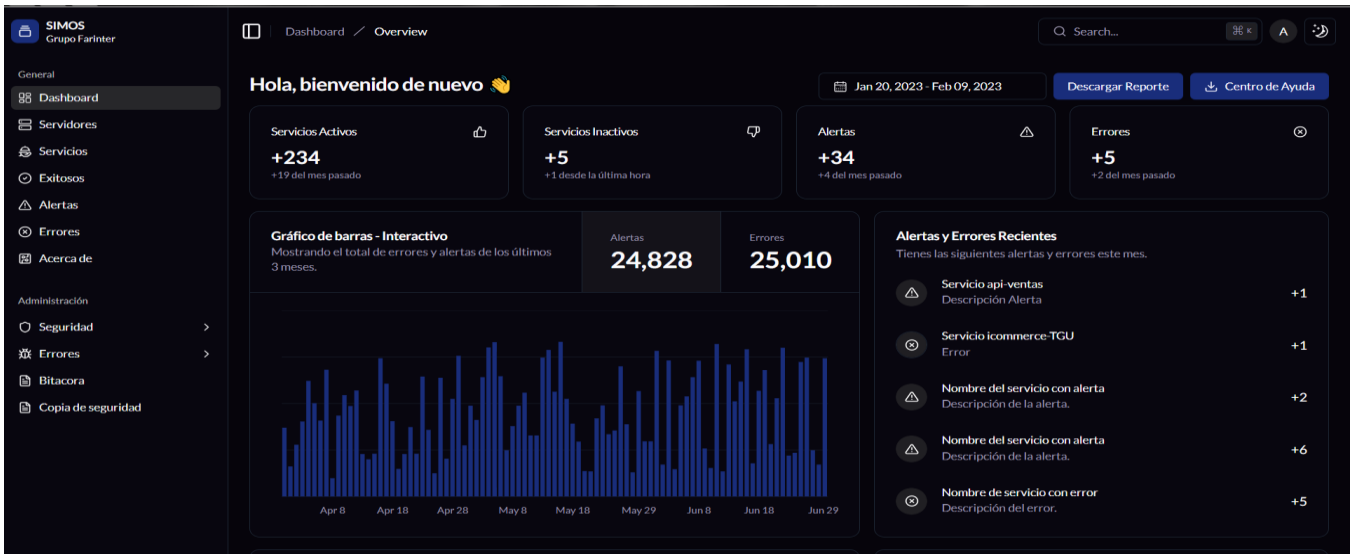


FIGURA 53 - SERVIDOR LEVANTADO

8.3 GLOSARIO.

1. **Dashboard:** Interfaz gráfica que muestra de manera organizada y visual información clave y métricas relevantes de un sistema o aplicación, facilitando el monitoreo y la toma de decisiones.
2. **Backup:** Proceso de crear una copia de seguridad de datos para protegerlos contra pérdida, corrupción o eliminación accidental.
3. **Restore:** Proceso de recuperar datos desde una copia de seguridad para restaurar un sistema o base de datos a un estado previo.
4. **Base de datos:** Conjunto organizado de información almacenada electrónicamente que puede ser accedida y gestionada mediante sistemas de gestión de bases de datos (SGBD).
5. **PostgreSQL:** Sistema de gestión de bases de datos relacional y de código abierto, conocido por su robustez y soporte a características avanzadas.
6. **npm run build:** Es un comando usado en proyectos de JavaScript, especialmente aquellos que utilizan Node.js y gestores de paquetes como **npm** (Node Package Manager).
7. **ORM:** Es un puente entre la base de datos relacional y el código del programa.
8. **.env:** Un archivo **.env** (abreviatura de *environment*) es un archivo de texto plano que contiene variables de entorno para configurar un proyecto o aplicación.
9. **Frontend:** Se refiere a la parte de una aplicación o sitio web con la que los usuarios interactúan directamente. Incluye todo lo que se ve en la pantalla, como el diseño, los botones, menús, textos, imágenes y la experiencia visual en general.
10. **Backend:** Es la parte de una aplicación o sistema que funciona detrás de escena y que no es visible para los usuarios. Se encarga de la lógica de negocio, el manejo de datos, la comunicación con bases de datos, la autenticación, y la gestión de solicitudes que vienen desde el frontend.
11. **Clone Repository:** Es el proceso de crear una copia local exacta de un repositorio de código fuente que está almacenado en un servidor remoto, como GitHub, GitLab o Bitbucket.
12. **Buffer:** es una región temporal de memoria utilizada para almacenar datos mientras se transfieren de un lugar a otro. Actúa como un área intermedia para manejar diferencias en la velocidad de procesamiento o en la tasa de transferencia entre dispositivos o procesos.
13. **VPS:** Es un servidor virtual privado que se crea al dividir un servidor físico en múltiples servidores virtuales independientes mediante tecnología de virtualización.
14. **Logs:** son registros automáticos que generan los sistemas, aplicaciones o servidores para documentar eventos, acciones o sucesos que ocurren durante su funcionamiento.
15. **SQL File:** Es un archivo que contiene código SQL (Structured Query Language), el lenguaje estándar para gestionar y manipular bases de datos relacionales.

Fin del manual

"Comprometidos con tu experiencia."

